

**SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE**

**Fakulta elektrotechniky a informatiky**

Evidenčné číslo: FEI-16605-126833

**Aplikácia na archiváciu a analýzu elektronických  
pokladničných blokov**

**Bakalárska práca**

**Bratislava 2026**

**Richard Laktiš**

**SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE**

**Fakulta elektrotechniky a informatiky**

Evidenčné číslo: FEI-16605-126833

# **Aplikácia na archiváciu a analýzu elektronických pokladničných blokov**

**Bakalárska práca**

|                      |                                        |
|----------------------|----------------------------------------|
| Študijný program:    | aplikovaná informatika                 |
| Študijný odbor:      | informatika                            |
| Školiace pracovisko: | Ústav informatiky a matematiky         |
| Školiteľ:            | Mgr. Lucia Pospíšilová Škripková, PhD. |

**Bratislava 2026**

**Richard Laktiš**



## ZADANIE BAKALÁRSKEJ PRÁCE

Študent: **Meno Priezvisko**  
ID študenta: 012345  
Študijný program: jadrové a fyzikálne inžinierstvo  
Študijný odbor: elektrotechnika  
Vedúci práce: tituly, Meno Priezvisko, tituly  
Vedúci pracoviska: tituly Meno Priezvisko, tituly  
Miesto vypracovania: Ústav jadrového a fyzikálneho inžinierstva

Názov práce: **Rozšírená šablóna záverečnej práce na FEI STU v Bratislave  
v systéme LaTeX**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

- Vytvorte šablónu záverečnej práce pre študentov bakalárskeho a inžinierskeho štúdia na FEI STU v Bratislave.
- Napište návod na písanie záverečnej práce s použitím vytvorenej šablóny.
- Vytvorte prehľad najpoužívanejších typov citovaných zdrojov a zdokumentujte ich tak, aby boli v súlade s normou ISO 690.

Zoznam odbornej literatúry:

- Zákon č. 131/2002 Z. z. z 21. februára 2002 o vysokých školách a o zmene a doplnení niektorých zákonov [online]. [cit. 2024-09-01]. Dostupné na [https://www.slov-lex.sk/static/pdf/2002/131/ZZ\\_2002\\_131\\_20240901.pdf](https://www.slov-lex.sk/static/pdf/2002/131/ZZ_2002_131_20240901.pdf)
- Vyhláška č. 233/2011 Z. z. Ministerstva školstva, vedy, výskumu a športu Slovenskej republiky z 1. júla 2011, ktorou sa vykonávajú niektoré ustanovenia zákona č. 131/2002 Z. z. o vysokých školách a o zmene a doplnení niektorých zákonov v znení neskorších predpisov [online]. [cit. 2024-09-01]. Dostupné na [https://www.slov-lex.sk/static/pdf/2011/233/ZZ\\_2011\\_233\\_20201015.pdf](https://www.slov-lex.sk/static/pdf/2011/233/ZZ_2011_233_20201015.pdf)

3. Metodické usmernenie Ministerstva školstva, vedy, výskumu a športu SR č. 56/2011 o náležitostiach záverečných prác, ich bibliografickej registrácii, kontrole originality, uchovávaní a sprístupňovaní č. 2011-11513/31015:4-071 [online]. [cit. 2024-09-24]. Dostupné na <http://www.minedu.sk/metodicke-usmernenie-c-562011-o-nalezitostiach-zaverecnych-prac-ich-bibliografickej-registracii-uchovavani-a-spristupnovani/>.
4. STN ISO 214: 1998. Dokumentácia. Abstrakty (referáty) pre publikácie a dokumentáciu. Bratislava: Slovenský ústav technickej normalizácie. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
5. STN ISO 690: 2022. Dokumentácia – Bibliografické odkazy – Obsah, forma a štruktúra. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
6. STN 01 6910: 2023. Pravidlá písania a úpravy písomností. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
7. STN ISO 2145: 1997. Dokumentácia. Číslovanie oddielov a pododdielov písaných dokumentov. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
8. STN ISO EN 80 000-1: 2022. Veličiny a jednotky. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
9. LICHNEROVÁ, L. a B. BELLÉROVÁ. Nové pravidlá citovania podľa ISO 690 z roku 2021. *ITlib. Informačné technológie a knižnice*, 3–4/2023, 10 – 22. [cit. 2024-09-06]. Dostupné na <http://doi.org/10.52036/1335793X.2023.3-4.10-22>.
10. HOFTICH, M. ISO 690 biblatex style [online]. [cit. 2024-09-30]. Dostupné na <https://mirrors.ibiblio.org/CTAN/macros/latex/contrib/biblatex-contrib/biblatex-iso690/biblatex-iso690.pdf>.

|                                             |                                                             |
|---------------------------------------------|-------------------------------------------------------------|
| Termín odovzdania bakalárskej práce:        | 31. 05. 2025                                                |
| Dátum schválenia zadania bakalárskej práce: | 28. 02. 2025                                                |
| Zadanie bakalárskej práce schválil:         | tituly Meno Priezvisko, tituly – garant študijného programu |

## Podakovanie

Na tomto mieste by som sa rád poďakoval prof. Ing. Davidovi Listerovi, DrSc. za cenné rady a pomoc pri písaní bakalárskej práce. Práca bola vypracovaná s podporou grantu Ministerstva školstva a vedy KEGA XXXXX/2022.

# Abstrakt

## Kľúčové slová

záverečná práca, šablóna, L<sup>A</sup>T<sub>E</sub>X, formátovanie textu, citácie

# Abstract

## Keywords

Final thesis, template, L<sup>A</sup>T<sub>E</sub>X, text formatting, citations

# Obsah

|                                                                |          |
|----------------------------------------------------------------|----------|
| Úvod                                                           | 1        |
| <b>1 Úvod do problematiky práce</b>                            | <b>2</b> |
| 1.1 Motivácia práce . . . . .                                  | 2        |
| 1.2 Cieľ práce . . . . .                                       | 2        |
| 1.3 Stručný opis riešenia . . . . .                            | 3        |
| 1.4 Štruktúra práce . . . . .                                  | 3        |
| <b>2 Teoretické východiská a analýza problematiky</b>          | <b>5</b> |
| 2.1 Pokladničné bloky . . . . .                                | 5        |
| 2.1.1 Papierové pokladničné bloky . . . . .                    | 5        |
| 2.1.2 Elektronické pokladničné bloky . . . . .                 | 6        |
| 2.2 Možnosti získavania údajov z blokov . . . . .              | 6        |
| 2.2.1 Manuálny vstup . . . . .                                 | 6        |
| 2.2.2 PDF bloky . . . . .                                      | 6        |
| 2.2.3 Obrázky blokov . . . . .                                 | 6        |
| 2.2.4 E-mailové bloky . . . . .                                | 7        |
| 2.3 Možnosti spracovania údajov z blokov . . . . .             | 7        |
| 2.3.1 Offline parsovanie pomocou regulárnych výrazov . . . . . | 7        |
| 2.3.2 OCR (Optical Character Recognition) . . . . .            | 8        |
| 2.3.3 Umelá inteligencia – jazykové modely . . . . .           | 8        |
| 2.3.4 Hybridný prístup . . . . .                               | 8        |
| 2.4 Technológie používané pri vývoji . . . . .                 | 9        |
| 2.4.1 Python . . . . .                                         | 9        |
| 2.4.2 Flask . . . . .                                          | 9        |
| 2.4.3 FastAPI . . . . .                                        | 10       |
| 2.4.4 HTML, CSS, JavaScript, Bootstrap . . . . .               | 10       |
| 2.4.5 SQLite . . . . .                                         | 10       |
| 2.4.6 IMAP . . . . .                                           | 11       |
| 2.4.7 REST API . . . . .                                       | 11       |
| 2.4.8 ngrok . . . . .                                          | 11       |
| 2.5 Bezpečnostné aspekty . . . . .                             | 11       |
| 2.5.1 Hashovanie hesiel . . . . .                              | 12       |
| 2.5.2 Šifrovanie citlivých údajov . . . . .                    | 12       |
| 2.5.3 Správa relácií (Sessions) . . . . .                      | 12       |
| 2.5.4 Validácia a kontrola vstupov . . . . .                   | 13       |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| 2.5.5    | Ochrana pred injekčnými útokmi (SQL injection) | 13        |
| <b>3</b> | <b>Analýza existujúcich riešení</b>            | <b>15</b> |
| 3.1      | Prehľad existujúcich riešení                   | 15        |
| 3.1.1    | eBločky                                        | 15        |
| 3.1.2    | E-blok                                         | 15        |
| 3.1.3    | Aplikácie obchodných reťazcov                  | 16        |
| 3.1.4    | Expensify                                      | 16        |
| 3.1.5    | Smart Receipts                                 | 16        |
| 3.2      | Porovnanie existujúcich riešení                | 17        |
| 3.3      | Požiadavky vyplývajúce z analýzy               | 17        |
| <b>4</b> | <b>Návrh riešenia</b>                          | <b>19</b> |
| 4.1      | Požiadavky na systém                           | 19        |
| 4.1.1    | Funkcionálne požiadavky                        | 19        |
| 4.1.2    | Nefunkcionálne požiadavky                      | 20        |
| 4.2      | Prípady použitia                               | 20        |
| 4.3      | Architektúra systému                           | 21        |
| 4.4      | Návrh databázy                                 | 23        |
| 4.5      | Návrh používateľského rozhrania                | 25        |
| 4.6      | Návrh spracovania blokov                       | 26        |
| 4.6.1    | Spracovanie PDF bloku                          | 27        |
| 4.6.2    | Spracovanie obrázku bloku                      | 28        |
| 4.6.3    | Import z e-mailovej schránky                   | 28        |
| 4.7      | Kategorizácia položiek                         | 29        |
| 4.8      | Návrh externého servera                        | 29        |
| <b>5</b> | <b>Implementácia aplikácie</b>                 | <b>31</b> |
| 5.1      | Štruktúra projektu                             | 31        |
| 5.2      | Použité knižnice                               | 32        |
| 5.3      | Autentifikácia a správa používateľov           | 34        |
| 5.4      | Databázová vrstva                              | 35        |
| 5.5      | Offline spracovanie PDF                        | 36        |
| 5.6      | Spracovanie pomocou umelej inteligencie        | 37        |
| 5.7      | Externý server                                 | 38        |
| 5.8      | Import z e-mailu                               | 38        |
| 5.9      | Archivácia súborov                             | 39        |
| 5.10     | Štatistiky a grafy                             | 39        |

|                                                    |           |
|----------------------------------------------------|-----------|
| 5.11 Používateľské rozhranie . . . . .             | 40        |
| <b>6 Testovanie a vyhodnotenie</b>                 | <b>44</b> |
| 6.1 Ciele a prostredie testovania . . . . .        | 44        |
| 6.2 Testovanie funkcií aplikácie . . . . .         | 44        |
| 6.3 Testovanie na reálnych bločkoch . . . . .      | 44        |
| 6.4 Vyhodnotenie výsledkov . . . . .               | 44        |
| 6.5 Porovnanie s existujúcimi riešeniami . . . . . | 44        |
| 6.6 Obmedzenia riešenia . . . . .                  | 44        |
| 6.7 Možnosti rozšírenia . . . . .                  | 44        |
| <b>Záver</b>                                       | <b>45</b> |
| <b>Literatúra</b>                                  | <b>47</b> |
| Použitie nástrojov umelej inteligencie . . . . .   | 50        |
| <b>A Algoritmus</b>                                | <b>51</b> |
| <b>B Výpis dlhého kódu</b>                         | <b>52</b> |
| <b>C Slovníček pojmov</b>                          | <b>53</b> |

# Zoznam obrázkov a tabuliek

|           |                                                                                                                                                                       |    |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Obrázok 1 | Diagram prípadov použitia . . . . .                                                                                                                                   | 21 |
| Obrázok 2 | Diagram komponentov a ich komunikácia . . . . .                                                                                                                       | 23 |
| Obrázok 3 | Logický model entitno-relačnej databázy . . . . .                                                                                                                     | 24 |
| Obrázok 4 | Návrh používateľského rozhrania . . . . .                                                                                                                             | 26 |
| Obrázok 5 | Diagram aktivít – tok spracovania PDF bloku . . . . .                                                                                                                 | 28 |
| Obrázok 6 | Obrazovka prihlásenia a registrácie . . . . .                                                                                                                         | 40 |
| Obrázok 7 | Hlavná obrazovka . . . . .                                                                                                                                            | 42 |
| Obrázok 8 | Obrazovka zoznamu blokov . . . . .                                                                                                                                    | 43 |
| Tabuľka 1 | Porovnanie prístupov spracovania pokladničných blokov . . .                                                                                                           | 9  |
| Tabuľka 2 | Porovnanie existujúcich riešení. *Funkcie umelej inteligencie v navrhovanej aplikácii využívajú služby tretích strán a vyžadujú vlastný API kľúč používateľa. . . . . | 17 |
| Tabuľka 3 | Endpointy externého servera . . . . .                                                                                                                                 | 30 |
| Tabuľka 4 | Knižnice hlavnej aplikácie . . . . .                                                                                                                                  | 33 |
| Tabuľka 5 | Knižnice externého servera . . . . .                                                                                                                                  | 33 |
| Tabuľka 6 | Frontend knižnice . . . . .                                                                                                                                           | 34 |

# Zoznam značiek a skratiek

PDF – prenosný formát dokumentov, z angl. *Portable Document Format*

HTML – značkovací jazyk pre tvorbu webových stránok, z angl. *HyperText Markup Language*

CSS – štýlovací jazyk pre úpravu vzhľadu webových stránok, z angl. *Cascading Style Sheets*

AI – umelá inteligencia, z angl. *Artificial Intelligence*

API – aplikačné programové rozhranie, z angl. *Application Programming Interface*

OCR – optické rozpoznávanie znakov, z angl. *Optical Character Recognition*

LLM – veľký jazykový model, z angl. *Large Language Model*

JSON – textový formát pre výmenu dát, z angl. *JavaScript Object Notation*

MLLM – multimodálny veľký jazykový model, z angl. *Multimodal Large Language Model*

# Zoznam algoritmov

# Zoznam výpisov kódov

# Úvod

# 1 Úvod do problematiky práce

## 1.1 Motivácia práce

V posledných rokoch dochádza v maloobchodnom sektore k postupnému prechodu od papierových pokladničných blokov k ich elektronickej forme. Hlavnými dôvodmi tohto trendu sú zníženie environmentálnej záťaže, znižovanie nákladov na strane obchodníkov a väčšie pohodlie pre zákazníkov [1]. Podľa správy Skip the Slip organizácie Green America emisie skleníkových plynov spojené s výrobou pokladničných blokov v Spojených štátoch amerických zodpovedajú ročným emisiám viac než 471 000 automobilov [2]. Elektronické pokladničné bloky predstavujú alternatívu, ktorá odstraňuje materiálovú produkciu a sú používateľom najčastejšie doručované prostredníctvom e-mailovej správy vo forme prílohy vo formáte PDF (Portable Document Format).

V prípade klasických papierových pokladničných blokov vzniká viacero praktických problémov, ktorým sa podrobnejšie venuje kapitola 2.1.

Napriek tomu, že elektronické pokladničné bloky predstavujú modernejšiu a ekologickjšiu alternatívu, ich praktické využitie v každodennom živote používateľa je stále obmedzené. Bloky sú často roztrúsené medzi e-mailovou schránkou, súbormi v zariadení alebo fotografiami v mobilnom telefóne, čo výrazne sťažuje ich prehľadné usporiadanie a následnú analýzu.

Používatelia tak nemajú k dispozícii jednotný nástroj, ktorý by im umožnil efektívne sledovať svoje výdavky na základe reálnych nákupných dát. Existujú aj manuálne riešenia, ako je zapisovanie výdavkov do tabuliek, avšak tento spôsob je časovo náročný a v praxi často neudržateľný.

Z uvedených dôvodov vzniká potreba nástroja, ktorý by proces archivácie a analýzy elektronických pokladničných blokov výrazne zjednodušil a zautomatizoval.

## 1.2 Cieľ práce

Cieľom tejto bakalárskej práce je návrh a implementácia lokálnej webovej aplikácie určenej na archiváciu a analýzu elektronických pokladničných blokov.

Hlavnou úlohou aplikácie je umožniť používateľovi získavanie a spracovanie informácií z pokladničných blokov viacerými spôsobmi – nahratím súboru vo formáte PDF, nahratím fotografie bloku, manuálnym zadáním údajov, ako aj automatizovaným prehľadávaním a importom blokov z e-mailovej schránky.

Spracovanie blokov je realizované hybridným prístupom, ktorý kombinuje offline parsovanie pomocou regulárnych výrazov s moderným prístupom využívajúcim umelú

inteligenciu v online režime. Súčasťou riešenia je aj návrh systému na archiváciu blokov, generovanie štatistík výdavkov a automatickú kategorizáciu jednotlivých položiek.

Funkčnosť a prínos aplikácie sú overené testovaním na reálnych elektronických pokladničných blokoch slovenských obchodných reťazcov.

### 1.3 Stručný opis riešenia

Aplikácia je implementovaná ako lokálna webová aplikácia distribuovaná vo forme spustiteľného súboru. Backend aplikácie je realizovaný v jazyku Python s využitím webového frameworku Flask. Používateľské rozhranie je tvorené štandardnými webovými technológiami (HTML, CSS, JavaScript). Na ukladanie dát aplikácia využíva relačnú databázu SQLite, ktorá je vzhľadom na lokálny charakter aplikácie a požiadavku na jednoduchú distribúciu vhodnejšia ako klasický databázový server. Spracovanie blokov prebieha hybridným spôsobom. V offline režime sú bloky spracovávané vlastnými parsermi založenými na regulárnych výrazoch. Pri zlyhaní offline spracovania alebo pri vstupoch, ktoré ho nedovoľujú (napríklad fotografie blokov), je využitý externý server sprostredkujúci komunikáciu so službou umelej inteligencie. Tento server je implementovaný ako samostatná komponenta nad frameworkom FastAPI a sprístupnený prostredníctvom nástroja ngrok, čím zabezpečuje, že prístupový kľúč k službe umelej inteligencie zostáva uložený mimo distribuovanej aplikácie. Aplikácia ďalej poskytuje import blokov z e-mailovej schránky používateľa, automatickú kategorizáciu položiek a prehľadné štatistiky výdavkov za zvolené časové obdobie.

### 1.4 Štruktúra práce

Bakalárska práca je členená do šiestich kapitol. Prvá kapitola sa zaoberá motiváciou, cieľom a stručným opisom riešenia.

Druhá kapitola sa sústreďuje na teoretické východiská práce. Opisuje pokladničné bloky, možnosti získavania a spracovania údajov z nich, použité technológie a bezpečnostné aspekty riešenia.

Tretia kapitola analyzuje existujúce riešenia v oblasti archivácie, analýzy a správy výdavkov, sumarizuje ich klady a nedostatky a identifikuje požiadavky na vlastné riešenie.

Štvrtá kapitola predstavuje podrobný návrh architektúry aplikácie, návrh databázy, používateľského rozhrania a jednotlivých spôsobov spracovania pokladničných blokov.

Piata kapitola obsahuje detailný opis štruktúry projektu, použitých knižníc a databázovej vrstvy. Zaoberá sa implementáciou offline spracovania blokov, jeho rozšírením o

online zálohu prostredníctvom umelej inteligencie a dodatočné funkcie, ako napríklad automatickou kategorizáciou položiek alebo import blokov z e-mailovej schránky.

V šiestej kapitole sú testované a vyhodnotené funkcionality aplikácie na reálnych pokladničných blokoch slovenských obchodných reťazcov, spolu s identifikáciou obmedzení a možností ďalšieho rozšírenia.

## 2 Teoretické východiská a analýza problematiky

Táto kapitola približuje problematiku pokladničných blokov, ich formy a spôsoby získavania a spracovania údajov. Zároveň predstavuje technológie, na ktorých je založené navrhované riešenie.

### 2.1 Pokladničné bloky

Pokladničný blok je doklad vydaný obchodníkom pri realizácii obchodnej transakcie, ktorý potvrdzuje jej uskutočnenie. Podľa zákona č. 289/2008 Z. z. o používaní elektronickej registračnej pokladnice musí pokladničný blok obsahovať povinné náležitosti, ako sú názov a adresa predajcu, dátum a čas uskutočnenia danej platby, zoznam zakúpených položiek vrátane cien, výšku dane z pridanej hodnoty a celkovú sumu nákupu [3]. Spôsob uchovávania a spracovania pokladničných blokov má významný vplyv na ich praktické využitie, najmä pri návrhu systémov na ich archiváciu a analýzu.

#### 2.1.1 Papierové pokladničné bloky

Papierové pokladničné bloky sú tlačené prostredníctvom tepelnej tlače na špeciálny termopapier, ktorý reaguje na pôsobenie tepla zmenou farby, čím umožňuje vytvorenie textu bez potreby použitia atramentu [4]. Tento spôsob tlače je rýchly a ekonomický, a preto sa široko využíva v maloobchodoch, veľkoobchodoch aj v rôznych podnikateľských prevádzkach.

Napriek širokému využitiu existuje viacero nevýhod. Pokladničné bloky bývajú často ukladané na rôzne miesta, čo vedie k ich postupnému hromadeniu a následnej neprehľadnosti, pričom vyhľadanie konkrétneho pokladničného bloku medzi väčším množstvom blokov je časovo náročné a neefektívne. Papierové pokladničné bloky sú tiež náchylné na vonkajšie vplyvy, ako sú vlhkosť, slnečné žiarenie alebo mechanické trenie, čo vedie k ich poškodeniu a postupnému blednutiu tlače. V dôsledku toho sa môže pokladničný blok stať nečitateľným a stratiť svoju hodnotu ako doklad o kúpe. Okrem toho majú papierové bloky spravidla krátku životnosť. V mnohých prípadoch si ich zákazníci neprevezmú, prípadne ich krátko po prevzatí vyhodia [5].

Podľa správy Skip the Slip organizácie Green America bola v roku 2021 v Spojených štátoch amerických na výrobu pokladničných blokov spotrebovaná drevná hmota zodpovedajúca približne 3 680 000 stromom a viac ako 10 miliardám galónov vody [2].

### **2.1.2 Elektronické pokladničné bloky**

Alternatívou ku klasickým papierovým pokladničným blokom sú elektronické pokladničné bloky, ktoré predstavujú digitálnu formu dokladu o realizovanej transakcii. Ich využitie umožňuje zníženie spotreby papiera, jednoduchosť doručenia a zlepšenie dostupnosti pre používateľa [6]. Tieto bloky sú používateľom najčastejšie doručované prostredníctvom e-mailovej schránky vo forme PDF, čo umožňuje mať k nim prístup kdekoľvek a kedykoľvek bez ohľadu na ich fyzickú formu, prípadne sú dostupné v rámci aplikácií špecifických pre konkrétny obchodný reťazec.

Napriek uvedeným výhodám však správa blokov rozptýlených v e-mailovej schránke bez systematickej organizácie prináša vlastné úskalia, čo môže viesť k ich neprehľadnosti a sťaženej práci pri ich vyhľadávaní a ďalšom spracovaní.

## **2.2 Možnosti získavania údajov z blokov**

Údaje z pokladničných blokov je možné získavať rôznymi spôsobmi. Tieto prístupy sa líšia mierou automatizácie a náročnosťou spracovania, čo ovplyvňuje ich praktické využitie.

### **2.2.1 Manuálny vstup**

Manuálnym vstupom sa rozumie zadávanie údajov z pokladničného bloku priamo používateľom, pričom jednotlivé informácie sú zadávané ručne do systému. Tento spôsob umožňuje dosahovať veľmi vysokú presnosť, keďže používateľ má plnú kontrolu nad zadávanými údajmi. Na druhej strane sa pri väčšom množstve dát stáva časovo náročným a prakticky ťažko realizovateľným.

### **2.2.2 PDF bloky**

Formát PDF predstavuje digitálny formát štandardizovaný medzinárodnou normou ISO 32000, určený na reprezentáciu dokumentov nezávisle od použitého softvéru, hardvéru alebo operačného systému. Umožňuje používateľom vymieňať a zobrazovať dokumenty spoľahlivo a nezávisle od prostredia, v ktorom boli vytvorené [7]. Pokladničné bloky v tomto formáte sú často strojovo čitateľné, čo umožňuje extrakciu textu priamo bez potreby rozpoznávania obrazu. Avšak nie všetky bloky vo formáte PDF sú textovo spracovateľné, keďže môžu namiesto textu obsahovať naskenovaný obraz.

### **2.2.3 Obrázky blokov**

Informácie z blokov môžu byť získavané aj z fotografií alebo skenov vo formátoch ako JPG, JPEG či PNG. Na rozdiel od textových PDF súborov však nie je možné z takýchto

blokov extrahovať text priamo, preto vyžadujú využitie dodatočných technológií, ktorým sa podrobnejšie venuje kapitola 2.3.2.

#### **2.2.4 E-mailové bloky**

Mnohé obchodné reťazce ponúkajú zasielanie elektronických pokladničných blokov priamo do e-mailovej schránky zákazníka vo forme prílohy vo formáte PDF. Prostredníctvom technológií, ktorým sa podrobnejšie venuje kapitola 2.4, je možné e-maily automatizovane filtrovať napríklad podľa odosielateľa alebo predmetu správy a stiahnuť príslušnú prílohu. Tento spôsob získavania blokov je vhodný na automatizované spracovanie bez nutnosti manuálneho sťahovania blokov používateľom.

Jednotlivé spôsoby získavania údajov sa líšia najmä mierou automatizácie a náročnosťou spracovania, čo priamo ovplyvňuje výber vhodnej metódy v konkrétnej aplikácii.

Z uvedených spôsobov získavania údajov vyplýva potreba navrhnúť riešenie, ktoré dokáže pracovať s rôznorodými vstupmi a zabezpečiť ich jednotné spracovanie. Táto požiadavka je zohľadnená aj v návrhu aplikácie.

### **2.3 Možnosti spracovania údajov z blokov**

Spracovanie údajov získaných z pokladničných blokov zahŕňa viacero prístupov s odlišnými vlastnosťami, výhodami a obmedzeniami.

#### **2.3.1 Offline parsovanie pomocou regulárnych výrazov**

Regulárne výrazy predstavujú formálnu notáciu na popis vzorov v textových reťazcoch, ktorá umožňuje vyhľadávanie, extrakciu a manipuláciu s textom, ktorý sa zhoduje s vopred definovaným vzorom. Získané textové reťazce je možné ďalej spracovať podľa potreby, napríklad zretaziť a tak vytvoriť nový textový reťazec [8]. Tento prístup je vhodný na extrakciu štrukturovaných informácií z blokov, ako sú názov obchodu, dátum a čas transakcie, názvy jednotlivých položiek a ich ceny. Výhodou tohto prístupu je vysoká rýchlosť, deterministickosť a skutočnosť, že nevyžaduje internetové pripojenie.

Na druhej strane, každý obchodný reťazec používa vlastný formát a štruktúru bloku, čo vyžaduje nutnosť vytvorenia samostatného parsera pre každý obchod zvlášť.

V kontexte tejto práce je tento prístup vhodný najmä pri spracovaní blokov so stabilnou štruktúrou, kde je možné presne definovať očakávané vzory.

### 2.3.2 OCR (Optical Character Recognition)

Nie všetky bloky vo formáte PDF sú však textovo čitateľné, niektoré obsahujú iba naskenovaný obraz, na ktorý nie je možné priamo aplikovať regulárne výrazy. V takomto prípade je možné využiť technológiu optického rozpoznávania znakov, označovanú ako OCR (Optical Character Recognition). Je to systém, ktorý nielen pomáha pri digitalizácii ručne písaných dokumentov, ale taktiež umožňuje aj konverziu obrazu textu do strojovo čitateľnej podoby [9]. Následná aplikácia regulárnych výrazov na takto získaný text dovoľuje celý tento prístup stále realizovať bez potreby internetového pripojenia.

Presnosť OCR je však závislá od kvality vstupného obrazu, čo zapríčiňuje nie vždy konzistentné výsledky. Faktory ako nedostatočné osvetlenie, nízke rozlíšenie alebo perspektívne deformácie môžu výrazne znížiť presnosť rozpoznávania [10]. Z uvedených dôvodov sa pri spracovaní fotografií alebo skenov textu často zvažujú alternatívne metódy spracovania, ktorým sa venuje nasledujúca kapitola.

### 2.3.3 Umelá inteligencia – jazykové modely

Veľké jazykové modely (Large Language Models, LLM) sú modely umelej inteligencie vopred natrénované na obrovskom množstve textových dát, ktoré sú schopné porozumieť prirodzenému jazyku a generovať text [11]. Rozšírením LLM sú tzv. multimodálne jazykové modely (Multimodal Large Language Models, MLLM), ktoré okrem textového vstupu podporujú aj spracovanie obrazových dát, na základe ktorých dokážu generovať príslušný textový výstup [12]. Pomocou vhodne definovaného promptu je možné model naviesť na generovanie štruktúrovaného výstupu, napríklad vo formáte JSON (JavaScript Object Notation), čo uľahčuje jeho ďalšie spracovanie.

V porovnaní s tradičnými metódami, ako sú regulárne výrazy alebo OCR, poskytujú jazykové modely vyššiu flexibilitu pri spracovaní rôznych formátov blokov, čo je výhodné najmä v prostredí s rôznorodými vstupmi.

Nevýhodou oproti klasickým metódam je nevyhnutnosť internetového pripojenia, vyššia výpočtová náročnosť, možnosť výskytu chýb, a taktiež náklady spojené s počtom spracovaných dát.

### 2.3.4 Hybridný prístup

Na základe vlastností jednotlivých prístupov je zrejmé, že žiadna metóda samostatne neposkytuje optimálne riešenie pre všetky typy vstupov. Z tohto dôvodu vzniká potreba ich kombinácie, čo vedie k využitiu hybridného prístupu. Hybridný prístup spočíva v kombinácii viacerých techník, ktoré umožňujú čiastočne eliminovať ich slabé stránky a

zároveň zachovať ich výhody. V kontexte spracovania pokladničných blokov môže zahŕňať napríklad kombináciu metód založených na regulárnych výrazoch a adaptívnych metód využívajúcich umelú inteligenciu [13]. Tento prístup umožňuje zachovanie funkčnosti aj bez internetového pripojenia, zníženie zaťaženia a zároveň flexibilitu pri spracovaní rôznych formátov vstupov. Podrobnejšie využitie hybridného prístupu je opísané v nasledujúcich kapitolách.

Pre lepšie porovnanie jednotlivých prístupov je ich stručný prehľad uvedený v tabuľke 1.

Tabuľka 1: Porovnanie prístupov spracovania pokladničných blokov

| Prístup            | Výhody                           | Nevýhody                         |
|--------------------|----------------------------------|----------------------------------|
| Regulárne výrazy   | Rýchle, deterministické, offline | Citlivé na zmeny formátu         |
| OCR                | Spracovanie obrázkov offline     | Nižšia presnosť pri zlej kvalite |
| Umelá inteligencia | Flexibilné, vyššia presnosť      | Vyžaduje internet, náklady       |
| Hybridný prístup   | Robustnosť, záložný mechanizmus  | Vyššia komplexnosť               |

## 2.4 Technológie používané pri vývoji

V tejto časti sú predstavené technológie použité pri vývoji navrhovanej aplikácie. Výber jednotlivých nástrojov reflektuje požiadavky na lokálne spracovanie dát, jednoduchosť nasadenia a možnosť ďalšieho rozšírenia systému. Nasledujúce podkapitoly stručne opisujú ich úlohu a využitie v rámci riešenia.

### 2.4.1 Python

Python je interpretovaný, objektovo orientovaný vysokoúrovňový programovací jazyk s pokročilými dátovými štruktúrami, ktorý kladie dôraz na čitateľnosť a jednoduchosť kódu. Disponuje rozsiahlou štandardnou knižnicou, ktorá pokrýva široké spektrum funkcionalít, ako sú práca so súborami, regulárne výrazy, prístup k databázam, podpora umelej inteligencie a mnoho ďalších. Vďaka svojej všestrannosti nachádza uplatnenie v rôznych oblastiach, napríklad pri vývoji aplikácií, spracovaní dát, automatizácii alebo pri vedeckých výpočtoch [14].

### 2.4.2 Flask

Pri vývoji webových aplikácií v jazyku Python sa široko využíva mikroframework Flask, ktorý poskytuje štandardné rozhranie medzi webovým serverom a webovou aplikáciou [15]. Obsahuje základné nástroje na smerovanie URL adries, spracovanie HTTP požiadaviek a odpovedí a vykresľovanie šablón. Na rozdiel od plnohodnotných

frameworkov Flask neobsahuje vstavanú podporu pre databázy, autentifikáciu ani iné komponenty, čo umožňuje vývojárovi väčšiu flexibilitu pri výbere nástrojov na tvorbu webových aplikácií.

### **2.4.3 FastAPI**

Na tvorbu vysokovýkonných API služieb v programovacom jazyku Python slúži moderný webový framework FastAPI. Využíva typové anotácie jazyka Python a štandardy ako OpenAPI a JSON Schema na definovanie a validáciu dát, čo umožňuje efektívnu komunikáciu medzi klientom a serverom [16]. FastAPI taktiež automaticky generuje interaktívnu dokumentáciu API, čím zjednodušuje vývoj a testovanie jednotlivých rozhraní.

### **2.4.4 HTML, CSS, JavaScript, Bootstrap**

Používateľské rozhranie webových aplikácií je tvorené kombináciou základných webových technológií ako HTML, CSS a JavaScript. HTML (HyperText Markup Language) je značkovací jazyk, ktorý definuje význam a štruktúru webového obsahu [17]. Na popis vizuálnej prezentácie dokumentov písaných v HTML sa používa štýlovací jazyk CSS (Cascading Style Sheets), ktorý definuje správanie a spôsob vykresľovania jednotlivých elementov na obrazovke [18]. JavaScript (JS) je interpretovaný programovací jazyk primárne využívaný na tvorbu interaktívnych a dynamických prvkov webových stránok na strane klienta [19].

Bootstrap je bezplatný open-source CSS framework určený na tvorbu responzívnych webových rozhraní. Poskytuje preddefinované komponenty a štýly ako sú navigačné prvky, tlačidlá a ďalšie, čo umožňuje rýchlu a jednotnú tvorbu používateľského rozhrania [20].

### **2.4.5 SQLite**

SQLite je odľahčený relačný databázový systém, ktorý na rozdiel od klasických databázových serverov funguje ako knižnica priamo integrovaná do aplikácie [21]. Kým databázové systémy typu klient-server, ako MySQL, Oracle alebo PostgreSQL, sú navrhnuté na správu zdieľaných dát s dôrazom na škálovateľnosť, súbežný prístup viacerých používateľov a centralizovanú kontrolu, SQLite sa zameriava na efektívnosť, spoľahlivosť a jednoduchosť použitia. Vďaka týmto vlastnostiam predstavuje vhodné riešenie pri lokálnom ukladaní dát pre jednotlivé aplikácie alebo zariadenia [22].

### 2.4.6 IMAP

Na vzdialený prístup k e-mailovým správam uloženým na poštovom serveri slúži protokol IMAP (Internet Message Access Protocol). Umožňuje prístup a správu poštovej schránky spôsobom, ktorý je funkčne ekvivalentný práci s lokálnymi priečkami. Protokol podporuje operácie ako vytváranie, mazanie a premenovávanie priečinkov poštovej schránky, vyhľadávanie nových správ a filtrovanie podľa definovaných parametrov [23]. Na základe týchto vlastností je IMAP využívaný v aplikáciách vyžadujúcich automatizovaný prístup k e-mailovým správam.

### 2.4.7 REST API

REST (Representational State Transfer) je architektonický štýl definovaný sadou obmedzení pre komunikáciu medzi klientom a serverom. Každá požiadavka od klienta smerom na server musí obsahovať všetky potrebné informácie na jej spracovanie, keďže server neuchováva stav medzi jednotlivými požiadavkami [24]. Na komunikáciu sa využívajú štandardné HTTP metódy. Najčastejšie používanou metódou je GET na získanie reprezentácie zdroja, POST na vytvorenie nového zdroja, PUT na úplnú aktualizáciu alebo nahradenie zdroja, PATCH na čiastočnú aktualizáciu a DELETE na jeho odstránenie. Metódy GET, PUT a DELETE sú zároveň idempotentné, čo znamená, že ich opakované volanie má rovnaký efekt na stav servera ako jediné volanie [25].

### 2.4.8 ngrok

Na sprístupnenie lokálne bežiacej aplikácie prostredníctvom verejnej URL adresy slúži nástroj ngrok, ktorý umožňuje prístup k lokálnym službám z internetu bez potreby vlastného servera [26]. Nástroj vytvára zabezpečený tunel medzi lokálnym serverom a ngrok serverom s takmer žiadnou manuálnou konfiguráciou sieťových prvkov ako sú DNS záznamy, IP adresy alebo certifikáty, keďže je spravovaná automaticky na strane služby. Ngrok zároveň dáva možnosť zabezpečiť prístup k aplikácii a vynútiť autentifikáciu, pričom zabráňuje obchádzaniu týchto mechanizmov priamym prístupom na lokálnu IP adresu [27].

Zvolené technológie umožňujú jednoduchú distribúciu aplikácie, lokálne spracovanie dát a zároveň minimalizujú závislosť od externých služieb. Táto kombinácia poskytuje efektívne a flexibilné riešenie, ktoré je možné v budúcnosti jednoducho rozširovať.

## 2.5 Bezpečnostné aspekty

Bezpečnosť hrá dôležitú úlohu pri návrhu webových aplikácií, najmä pri ochrane citlivých údajov a zabezpečení komunikácie medzi klientom a serverom. Nasledujúce podkapi-

toly sa venujú vybraným mechanizmom a technikám, ktoré prispievajú k bezpečnosti navrhovaného riešenia.

### 2.5.1 Hashovanie hesiel

Jedným z kľúčových bezpečnostných aspektov aplikácií pracujúcich s používateľskými účtami je ochrana citlivých údajov, ako sú heslá. Na tento účel sa využívajú moderné adaptívne hashovacie algoritmy. Hashovanie predstavuje jednosmernú kryptografickú funkciu, ktorá transformuje vstupný text ľubovoľnej veľkosti na reťazec pevnej dĺžky, pričom z výsledného výstupu nie je možné spätne získať pôvodný vstup. Na zvýšenie bezpečnosti sa ešte pred samotným zahashovaním používa tzv. salt. Ide o náhodne generovaný reťazec znakov jedinečný pre každého používateľa, ktorý je pridaný k heslu ako súčasť hashovacieho procesu [28]. Dôležitou vlastnosťou hashovania je deterministickosť, ktorá zaručuje, že rovnaký vstup vždy produkuje rovnaký výstup, čo je nevyhnutné pri overovaní hesla počas prihlasovania [29].

### 2.5.2 Šifrovanie citlivých údajov

Ďalšou metódou ochrany citlivých údajov je šifrovanie. Šifrovanie transformuje vstupné dáta do nečitateľnej formy – šifrovaného textu. Na rozdiel od hashovania je tento proces reverzibilný prostredníctvom dešifrovania pomocou kryptografického kľúča [30]. Je vhodné na ukladanie údajov, ktoré je neskôr potrebné zobrazit v pôvodnej forme, ako napríklad adresa používateľa. Naopak, šifrovanie hesiel by sa malo používať iba v okrajových prípadoch, kde je naozaj šifrované heslo nevyhnutné získať v pôvodnej forme, napríklad ak aplikácia vyžaduje heslo na autentifikáciu v inom systéme. V opačnom prípade by sa na zabezpečenie hesla malo vždy použiť hashovanie [28].

### 2.5.3 Správa relácií (Sessions)

HTTP je bezstavový protokol, kde každá požiadavka a odpoveď tvoria nezávislý pár komunikácie medzi klientom a serverom. Z tohto dôvodu je potrebné zabezpečiť mechanizmus, ktorý umožní serveru uchovávať stav používateľa medzi jednotlivými požiadavkami. Tento mechanizmus zabezpečujú relácie (sessions). Webová relácia predstavuje sekvenciu požiadaviek a odpovedí viazaných na konkrétneho používateľa počas jeho interakcie s aplikáciou. Pri vytvorení relácie server vygeneruje jedinečný identifikátor relácie (Session ID), ktorý je uložený v cookie vo webovom prehliadači a zdieľaný medzi klientom a serverom pri každej nasledujúcej požiadavke [31]. Na základe tohto identifikátora je server schopný jednoznačne identifikovať konkrétneho používateľa. Z bezpečnostného hľadiska je dôležité zabezpečiť ochranu identifikátora relácie, napríklad

prostredníctvom použitia zabezpečeného prenosu (HTTPS) alebo obmedzenia jeho platnosti, aby sa zabránilo jeho zneužitiu neoprávnenou osobou.

### 2.5.4 Validácia a kontrola vstupov

Validácia vstupov predstavuje neoddeliteľnú súčasť bezpečnej aplikácie, ktorej cieľom je zabezpečiť, aby do systému vstupovali iba dáta v správnom formáte a očakávanej štruktúre. Jej úlohou je predchádzať ukladaniu chybných údajov a zároveň zabráňovať spracovaniu škodlivých vstupov, ktoré by mohli narušiť funkčnosť systému. Validáciu vstupov je možné rozdeliť na syntaktickú a sémantickú.

Hlavnou funkciou syntaktickej validácie je zabezpečenie správneho formátu a štruktúry vstupných údajov, napríklad pri overovaní dátumu, e-mailovej adresy alebo číselných hodnôt.

Sémantická validácia sa zameriava na správnosť údajov v špecifickom kontexte, napríklad kontrolu rozsahu hodnôt alebo overenie logických závislostí a vzťahov medzi údajmi (napr. dátum konca transakcie nasleduje po dátume jej začiatku). Z bezpečnostného hľadiska je dôležité vykonávať validáciu vstupov čo najskôr v procese spracovania požiadavky. Týmto spôsobom je možné odhaliť neplatné alebo škodlivé vstupy ešte pred ich samotným spracovaním aplikáciou [32].

Validácia vstupov môže byť realizovaná na strane klienta (frontend) aj na strane servera (backend). Frontend validácia slúži najmä na zlepšenie používateľskej skúsenosti, keďže umožňuje okamžitú kontrolu vstupov a rýchlu spätnú väzbu bez potreby odoslania požiadavky na server. Na druhej strane, backend validácia predstavuje nevyhnutnú bezpečnostnú vrstvu, ktorá zabezpečuje kontrolu všetkých vstupných dát nezávisle od činnosti klienta. Keďže klientská časť aplikácie môže byť používateľom upravená alebo obídená, validácia na strane servera musí byť vždy implementovaná ako hlavný mechanizmus ochrany. Z tohto dôvodu sa v praxi odporúča kombinovať oba prístupy, pričom frontend validácia zvyšuje komfort používateľa a backend validácia zabezpečuje integritu a bezpečnosť spracovávaných dát [32].

### 2.5.5 Ochrana pred injekčnými útokmi (SQL injection)

Injekčný útok (SQL injection) je konkrétny typ útoku na aplikáciu, ktorá nemá dostatočne zabezpečenú validáciu vstupných dát pri vykonávaní SQL príkazov. Útočník vkladá škodlivý kód do vstupného poľa aplikácie s cieľom manipulovať s databázou. Úspešný útok môže viesť k neoprávnenému čítaniu údajov, úprave alebo vymazaniu dát, prípadne k úplnému prevzatiu kontroly nad databázou. Na prevenciu injekčných útokov existujú overené techniky aplikovateľné v akomkoľvek programovacom jazyku. Najefektívnejšou ochranou je použitie parametrizovaných dotazov (prepared statements), kde sú

vstupné hodnoty oddelené od samotnej štruktúry SQL príkazu a spracované samostatne. Ďalšou možnosťou je uplatnenie princípu najmenších oprávnení, pri ktorom má používateľ iba minimálne potrebné oprávnenia nevyhnutné pre chod aplikácie. Kombináciou týchto opatrení je možné výrazne znížiť riziko úspešného injekčného útoku [33].

## 3 Analýza existujúcich riešení

V tejto kapitole sú analyzované existujúce riešenia v oblasti správy a evidencie pokladničných blokov, sumarizované ich klady a nedostatky a identifikované požiadavky na navrhovanú aplikáciu.

### 3.1 Prehľad existujúcich riešení

Cieľom nasledujúcich podkapitol je predstaviť vybrané existujúce riešenia zamerané na správu pokladničných blokov a výdavkov. Dôraz je kladený na ich funkcionality, spôsob spracovania údajov a identifikáciu ich výhod a obmedzení.

#### 3.1.1 eBločky

Aplikácia eBločky je bezplatná slovenská mobilná aplikácia pre platformy Android a iOS určená na správu pokladničných blokov [34]. Každý pokladničný blok vydaný prostredníctvom systému eKasa obsahuje jedinečný QR kód, ktorým môže zákazník pokladničný doklad jednoducho overiť [35]. Práve tento QR kód aplikácia využíva na presné načítanie údajov z blokov. Okrem evidencie blokov ponúka doplnkové funkcie, ako je správa vernostných kariet alebo sledovanie záručných lehôt. Výhodou aplikácie je jednoduchosť používania a presnosť získaných údajov.

Na druhej strane je jej využitie obmedzené na bloky obsahujúce QR kód systému eKasa, čo obmedzuje spracovanie blokov z iných zdrojov alebo formátov.

#### 3.1.2 E-blok

Ďalším slovenským riešením je aplikácia E-blok, ktorá je určená na správu pokladničných blokov a faktúr, pričom je dostupná prostredníctvom mobilnej aplikácie aj webového rozhrania [36]. Umožňuje skenovanie QR kódov, rozpoznávanie zahraničných dokladov a faktúr, sledovanie výdavkov v grafoch a export blokov do formátu PDF.

Výhodou aplikácie je podpora zahraničných dokladov a faktúr a pokročilejšie štatistiky oproti konkurenčným riešeniam. Plný prístup ku všetkým funkciám je však podmienený predplatením, pričom bezplatná verzia v čase písania práce obmedzuje počet naskenovaných QR kódov na 150 za mesiac. Aplikácia je zároveň orientovaná najmä na spracovanie blokov prostredníctvom skenovania QR kódu a neumožňuje priame spracovanie dokumentov vo formáte PDF, čo môže obmedzovať jej využitie pri elektronických blokoch doručených e-mailom.

### 3.1.3 Aplikácie obchodných reťazcov

Viaceré slovenské obchodné reťazce poskytujú buď vlastné mobilné aplikácie, umožňujú zasielanie elektronických blokov priamo na e-mailovú adresu zákazníka alebo ich kombináciu. Napríklad reťazce Coop Jednota, Terno alebo Dr. Max umožňujú zasielanie blokov priamo na e-mailovú adresu zákazníka, zatiaľ čo iné reťazce ako dm či Kaufland poskytujú bloky výhradne prostredníctvom vlastných mobilných aplikácií, niektoré aj s možnosťou ich stiahnutia vo formáte PDF [37–39].

Nevýhodou tohto prístupu je skutočnosť, že zákazník nakupujúci vo viacerých obchodoch je nútený spravovať niekoľko samostatných aplikácií, čo vedie k neprehľadnosti a sťaženej evidencii celkových výdavkov naprieč rôznymi predajňami.

### 3.1.4 Expensify

Medzi zahraničné riešenia na správu výdavkov a účteniek patrí cloudová aplikácia Expensify, dostupná pre platformy Android, iOS aj ako webové rozhranie [40]. Umožňuje skenovanie blokov pomocou vlastnej technológie SmartScan, automatickú kategorizáciu výdavkov a generovanie výkazov. Aplikácia je primárne orientovaná na firemné prostredie, ponúka však aj plán pre individuálnych používateľov.

Tieto nástroje umožňujú efektívnu správu výdavkov, zdieľanie dát a generovanie výkazov. Na druhej strane ide o riešenie závislé od internetového pripojenia a cloudového prostredia, pričom bezplatná verzia v čase písania práce obmedzuje počet skenov na 25 za mesiac a plný prístup je podmienený predplattým. Aplikácia podporuje viaceré jazyky, slovenčina však medzi nimi nie je zahrnutá.

### 3.1.5 Smart Receipts

Ďalšou zahraničnou možnosťou je aplikácia Smart Receipts pre platformy Android, iOS aj ako webové rozhranie určená na správu a evidenciu účteniek a výdavkov [41]. Aplikácia disponuje rozsiahlymi funkciami ako skenovanie blokov pomocou OCR, import blokov z e-mailu, kategorizácia výdavkov a export do formátov PDF, CSV a ďalších.

Bezplatná verzia je však dostupná výhradne v mobilnej aplikácii a obmedzuje používateľa len na manuálny vstup blokov a základné operácie. Všetky pokročilé funkcie vrátane automatického skenovania, importu z e-mailu a exportu dát sú podmienené predplattým. Podobne ako pri aplikácii Expensify, kategorizácia prebieha na úrovni celého bloku, nie jednotlivých položiek.

### 3.2 Porovnanie existujúcich riešení

Na základe analýzy existujúcich riešení boli identifikované nedostatky v kľúčových vlastnostiach, podľa ktorých sú aplikácie porovnané v tabuľke 2.

Tabuľka 2: Porovnanie existujúcich riešení. \*Funkcie umelej inteligencie v navrhovanej aplikácii využívajú služby tretích strán a vyžadujú vlastný API kľúč používateľa.

| Vlastnosť                  | eBločky | E-blok      | Expensify   | Smart Receipts   | Navrhovaná aplikácia |
|----------------------------|---------|-------------|-------------|------------------|----------------------|
| Platforma                  | Mobilná | Mobilná/Web | Mobilná/Web | Mobilná/Web(Pro) | Web/Desktop          |
| Lokálne ukladanie          | Nie     | Nie         | Nie         | Nie              | Áno                  |
| Cloudová synchronizácia    | Áno     | Áno         | Áno         | Áno              | Nie                  |
| Import z e-mailu           | Nie     | Nie         | Nie         | Áno (Pro)        | Áno                  |
| PDF upload                 | Nie     | Nie         | Áno         | Áno (Pro)        | Áno                  |
| Offline spracovanie        | Nie     | Nie         | Nie         | Nie              | Čiastočne            |
| AI spracovanie             | Nie     | Áno         | Áno         | Áno (Pro)        | Áno                  |
| Kat. jednotlivých položiek | Nie     | Nie         | Nie         | Nie              | Áno                  |
| Štatistiky a grafy         | Áno     | Áno         | Áno         | Áno              | Áno                  |
| Slovenský jazyk            | Áno     | Áno         | Nie         | Áno              | Áno                  |
| Bezplatná plná verzia      | Áno     | Nie         | Nie         | Nie              | Áno*                 |

Z porovnania vyplýva, že väčšina analyzovaných riešení je závislá od cloudového prostredia, pričom dáta používateľa sú ukladané na vzdialených serveroch. Žiadne z analyzovaných riešení nepodporuje offline spracovanie, čo znamená, že hlavné funkcionality týchto aplikácií sú závislé od internetového pripojenia. Možnosť automatizovaného spracovania blokov z e-mailovej schránky je v analyzovaných riešeniach obmedzená a nie je bežne podporovaná. Zároveň žiadna z analyzovaných aplikácií neposkytuje automatickú kategorizáciu na úrovni jednotlivých položiek pokladničného bloku. Pokročilé funkcie väčšiny aplikácií sú navyše podmienené predplattým, čo obmedzuje ich dostupnosť pre bežného používateľa. Navrhovaná aplikácia rieši tieto nedostatky hybridným prístupom, ktorý umožňuje offline spracovanie podporovaných formátov a online spracovanie ako záložné riešenie, podporu kategorizácie na úrovni jednotlivých položiek a bezplatnú distribúciu.

### 3.3 Požiadavky vyplývajúce z analýzy

Na základe analýzy existujúcich riešení a ich nedostatkov boli identifikované nasledujúce požiadavky na navrhovanú aplikáciu:

- **Podpora viacerých vstupných formátov** – aplikácia musí umožňovať spracovanie blokov z rôznych zdrojov, ako sú PDF súbory, obrázky, e-mailové prílohy alebo manuálny vstup. Táto požiadavka vyplýva z povahy elektronických blokov, ktoré sú používateľom doručované rozličnými cestami.

- **Offline spracovanie** – aplikácia musí podporovať offline spracovanie blokov slovenských obchodných reťazcov, ktoré umožňujú zasielanie alebo export pokladničných blokov vo formáte PDF. Táto požiadavka znižuje závislosť aplikácie od externých služieb a internetového pripojenia.
- **Automatizovaný import z e-mailu** – aplikácia musí podporovať automatizovaný import blokov z e-mailovej schránky. Táto funkcionality šetrí používateľovi čas v porovnaní s manuálnym sťahovaním a nahrávaním jednotlivých blokov.
- **Lokálne ukladanie dát** – databáza aplikácie musí byť ukladaná lokálne na zariadení používateľa bez závislosti od cloudových služieb. Tým sa zachováva súkromie používateľa a kontrola nad jeho nákupnými údajmi, ktoré nie sú odovzdávané poskytovateľovi cloudovej služby. Databáza uchováva rozparované údaje z blokov, ako sú názov obchodu, dátum, položky a ich ceny. Ukladanie samotných súborov blokov na disk do samostatného priečinka je voliteľné, čím sa predchádza zbytočnému zaberaniu úložného priestoru.
- **Kategorizácia jednotlivých položiek** – aplikácia musí poskytovať kategorizáciu na úrovni jednotlivých položiek bloku, nie len celého nákupu. Detailnejšia kategorizácia umožňuje presnejšiu analýzu výdavkov, napríklad rozlíšenie potravín a drogérie v rámci jedného nákupu.
- **Bezplatná dostupnosť** – aplikácia musí byť distribuovaná bezplatne. Funkcionality využívajúca umelú inteligenciu vyžaduje prístupový kľúč k externej službe, mnohí poskytovatelia však ponúkajú bezplatné kvóty postačujúce pre bežné použitie.

Tieto požiadavky viedli k návrhu hybridného riešenia, ktoré kombinuje offline parsovanie s online spracovaním pomocou umelej inteligencie, čomu sa venujú nasledujúce kapitoly zamerané na návrh a implementáciu tohto prístupu.

## 4 Návrh riešenia

Na základe požiadaviek identifikovaných v predchádzajúcej kapitole je v tejto časti prezentovaný návrh aplikácie s pracovným názvom **Reccio**, jej architektúra, návrh databázy, používateľského rozhrania a spôsoby spracovania pokladničných blokov.

### 4.1 Požiadavky na systém

V tejto podkapitole sú definované požiadavky na aplikáciu Reccio, ktoré sú ďalej rozdelené na funkcionálne a nefunkcionálne.

#### 4.1.1 Funkcionálne požiadavky

Funkcionálne požiadavky definujú, čo aplikácia musí vedieť vykonať z pohľadu používateľa:

- **Registrácia a prihlásenie** – aplikácia musí umožňovať registráciu nového používateľa a prihlásenie pomocou mena a hesla.
- **Správa účtu** – používateľ musí mať možnosť zmeniť svoje používateľské meno, heslo a e-mailové nastavenia.
- **Import pokladničných blokov** – aplikácia musí umožňovať získavanie blokov viacerými spôsobmi: nahratím súboru vo formáte PDF, nahratím fotografie bloku, manuálnym zadaním údajov, ako aj automatizovaným importom z e-mailovej schránky s možnosťou filtrovania podľa odosielateľa alebo predmetu správy.
- **Kategorizácia položiek** – aplikácia musí automaticky priradovať kategórie jednotlivým položkám pokladničného bloku.
- **Prehľad a štatistiky výdavkov** – aplikácia musí zobrazovať prehľad výdavkov aktuálneho mesiaca, posledné a najdrahšie nákupy, ako aj podrobné štatistiky za zvolené časové obdobie vrátane grafov kategórií a obchodov.
- **Archivácia súborov** – aplikácia musí umožňovať voliteľné ukladanie súborov blokov na disk.
- **Detekcia duplícít** – aplikácia musí zabráňovať uloženiu duplicitných pokladničných blokov do databázy.

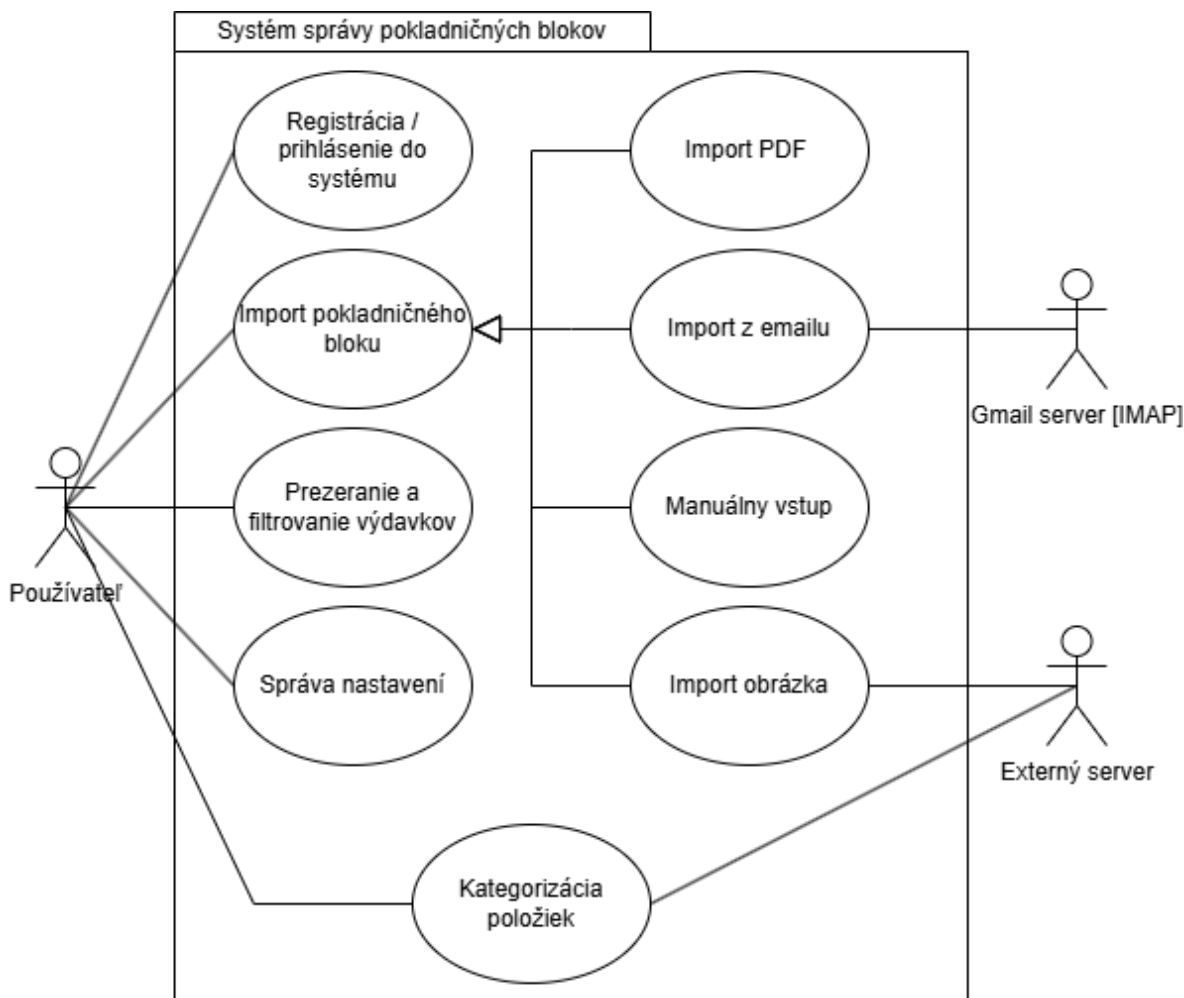
### 4.1.2 Nefunkcionálne požiadavky

Nefunkcionálne požiadavky definujú vlastnosti a obmedzenia systému, teda ako má aplikácia fungovať:

- **Použitelnosť** – používateľské rozhranie musí byť prehľadné a intuitívne.
- **Offline režim** – základné funkcie aplikácie musia byť dostupné bez internetového pripojenia.
- **Bezpečnosť** – heslá používateľov musia byť ukladané v podobe odolného hashu pomocou adaptívneho algoritmu (napríklad bcrypt alebo scrypt). Citlivé údaje, ktoré je nevyhnutné spätne dešifrovať (napríklad aplikačné heslo k e-mailovej schránke), musia byť ukladané v šifrovanej forme symetrickou kryptografickou metódou.
- **Jednoduchosť spustenia** – aplikácia by mala byť distribuovateľná vo forme spustiteľného súboru s minimálnymi požiadavkami na konfiguráciu používateľského prostredia.
- **Bezpečné spracovanie AI požiadaviek** – prístupový kľúč k službe umelej inteligencie nesmie byť uložený v distribuovanej klientskej aplikácii. Spracovanie požiadaviek musí prebiehať prostredníctvom samostatnej serverovej komponenty, ktorá tento kľúč udržiava výhradne na strane servera.
- **Modularita a rozšíriteľnosť** – aplikácia musí byť navrhnutá modulárne, aby bolo možné jednoducho doplniť parsery pre nové formáty blokov alebo nové zdroje vstupných dát.

## 4.2 Prípady použitia

Diagram prípadov použitia znázorňuje hlavné interakcie medzi používateľom a systémom. Systém komunikuje s dvoma externými aktérmi, s Gmail serverom prostredníctvom protokolu IMAP a externým serverom sprostredkujúcim komunikáciu so službami umelej inteligencie. Diagram je znázornený na obrázku 1.



Obr. 1: Diagram prípadov použitia

Identifikované prípady použitia pokrývajú všetky hlavné interakcie používateľa so systémom. Základným predpokladom používania aplikácie je registrácia a prihlásenie do systému. Po prihlásení môže používateľ pracovať s aplikáciou, spravovať osobné údaje a nastavenia nevyhnutné pre importovanie blokov z e-mailovej schránky. Aplikácia umožňuje nahrávanie blokov viacerými spôsobmi, napríklad vo formáte PDF, vo forme obrázkov alebo prostredníctvom manuálneho zadania údajov. V prípade, že položky neboli automaticky kategorizované, môže používateľ spustiť hromadnú kategorizáciu prostredníctvom externého servera využívajúceho umelú inteligenciu. Aplikácia ďalej umožňuje prezeranie štatistík, filtrovanie a vyhľadávanie uložených blokov.

### 4.3 Architektúra systému

Aplikácia je navrhnutá ako lokálna webová aplikácia, ktorá je spúšťaná na zariadení používateľa bez potreby nasadenia na vzdialený server. Architektúra systému je rozdelená

na niekoľko hlavných komponentov, ktoré spolu zabezpečujú spracovanie pokladničných blokov a poskytovanie funkcionality používateľovi.

Používateľ komunikuje s aplikáciou prostredníctvom webového rozhrania, ktoré je implementované pomocou technológií HTML, CSS a JavaScript. Toto rozhranie zabezpečuje zobrazovanie údajov, interakciu s používateľom a odosielanie požiadaviek na serverovú časť aplikácie. Serverová časť je implementovaná pomocou frameworku Flask, ktorý zabezpečuje smerovanie požiadaviek, spracovanie blokov a komunikáciu s databázou.

Na ukladanie dát je využívaná lokálna databáza SQLite, ktorá uchováva spracované údaje o pokladničných blokoch, ako sú názov obchodu, dátum nákupu, jednotlivé položky a ich ceny. Súborový blok sú voliteľne ukladané do súborového systému v samostatných priečiňkách podľa jednotlivých používateľov.

Spracovanie pokladničných blokov je realizované hybridným prístupom. V prípade podporovaných formátov blokov sú spracované offline pomocou parserov založených na regulárnych výrazoch. V opačnom prípade, napríklad pri spracovaní obrázkov, nepodporovanom formáte alebo pri hromadnej kategorizácii položiek je využitý externý server využívajúci umelú inteligenciu.

Pre automatizovaný import pokladničných blokov z e-mailovej schránky aplikácia komunikuje s Gmail serverom prostredníctvom protokolu IMAP.

Externý server je implementovaný ako samostatná komponenta pomocou frameworku FastAPI, ktorá slúži ako prostredník medzi hlavnou aplikáciou a službami umelej inteligencie tretích strán. Hlavná aplikácia odošle požiadavku na tento server, ktorý ju spracuje a odošle príslušný prompt službám umelej inteligencie, konkrétne OpenAI API. Výsledok je následne vrátený hlavnej aplikácii vo forme štruktúrovanej odpovede. Server je sprístupnený prostredníctvom nástroja ngrok, ktorý vytvára verejne dostupný tunel na lokálne bežiacu serverovú komponentu. Vďaka oddeleniu serverovej komponenty od klientskej aplikácie zostáva prístupový kľúč k službe umelej inteligencie uložený výhradne na strane servera.

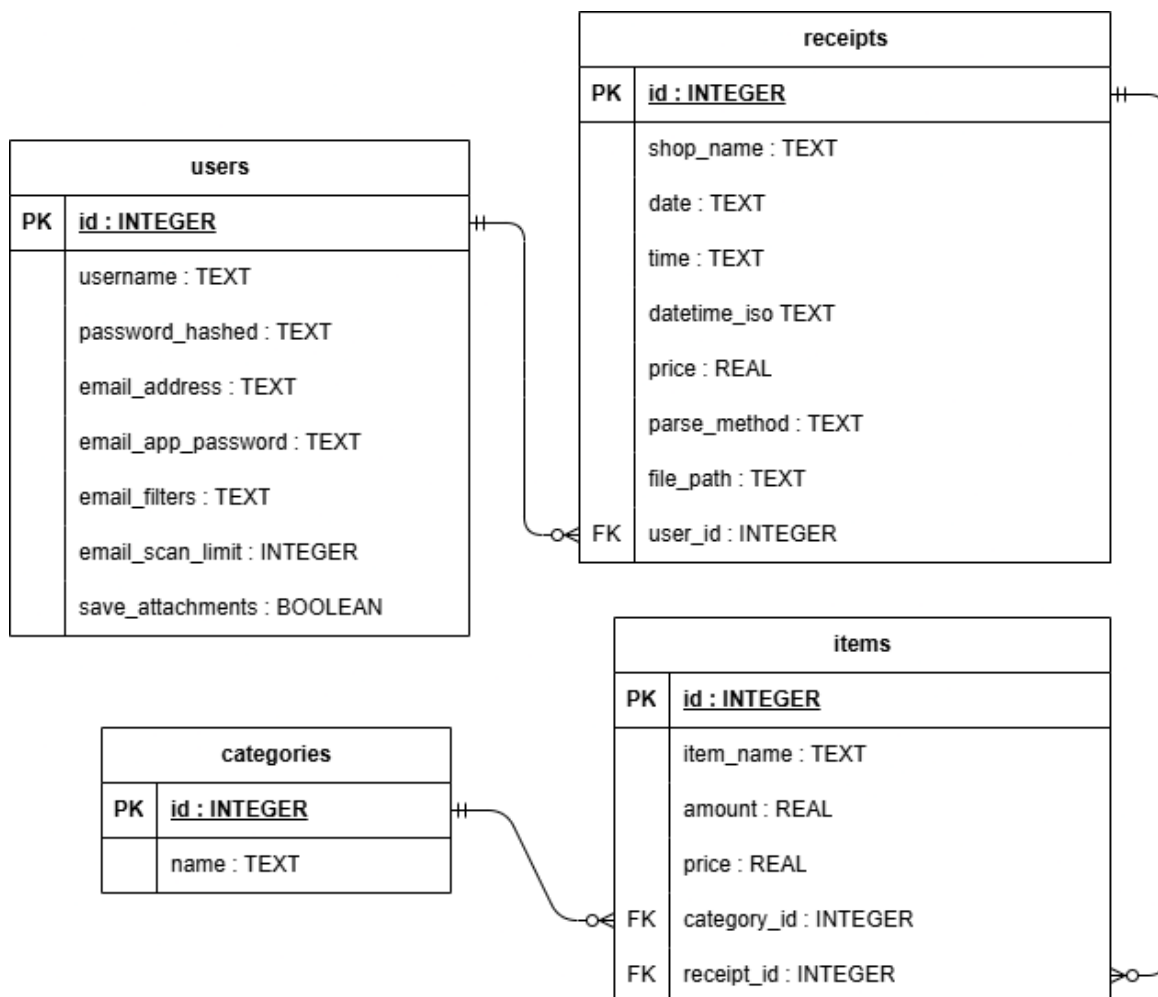
Navrhnutá architektúra oddeľuje používateľské rozhranie, aplikačnú logiku a spracovanie údajov, čím zvyšuje prehľadnosť systému a umožňuje jeho jednoduchšie rozšírenie a údržbu. Táto architektúra je znázornená na obrázku 2.



- **categories** – uchováva zoznam kategórií, do ktorých sú jednotlivé položky zaradované. Týmto návrhom sa predchádza redundantnému ukladaniu rovnakého názvu kategórie pri každej položke a zabezpečuje sa konzistencia kategorizácie.

Medzi jednotlivými tabuľkami existujú vzťahy typu 1:N. To znamená, že jeden používateľ môže mať uložených viacero pokladničných blokov, pričom každý pokladničný blok môže obsahovať viacero položiek. Jedna kategória môže byť priradená viacerým položkám.

Vzťah medzi tabuľkami *users* a *receipts* je realizovaný pomocou cudzieho kľúča *user\_id*, ktorý zabezpečuje prepojenie pokladničných blokov s konkrétnym používateľom. Podobne je vzťah medzi tabuľkami *receipts* a *items* realizovaný pomocou cudzieho kľúča *receipt\_id*. Vzťah medzi *items* a *categories* pomocou cudzieho kľúča *category\_id*. Podrobná štruktúra logického modelu relačnej databázy je znázornená na obrázku 3.



Obr. 3: Logický model entitno-relačnej databázy

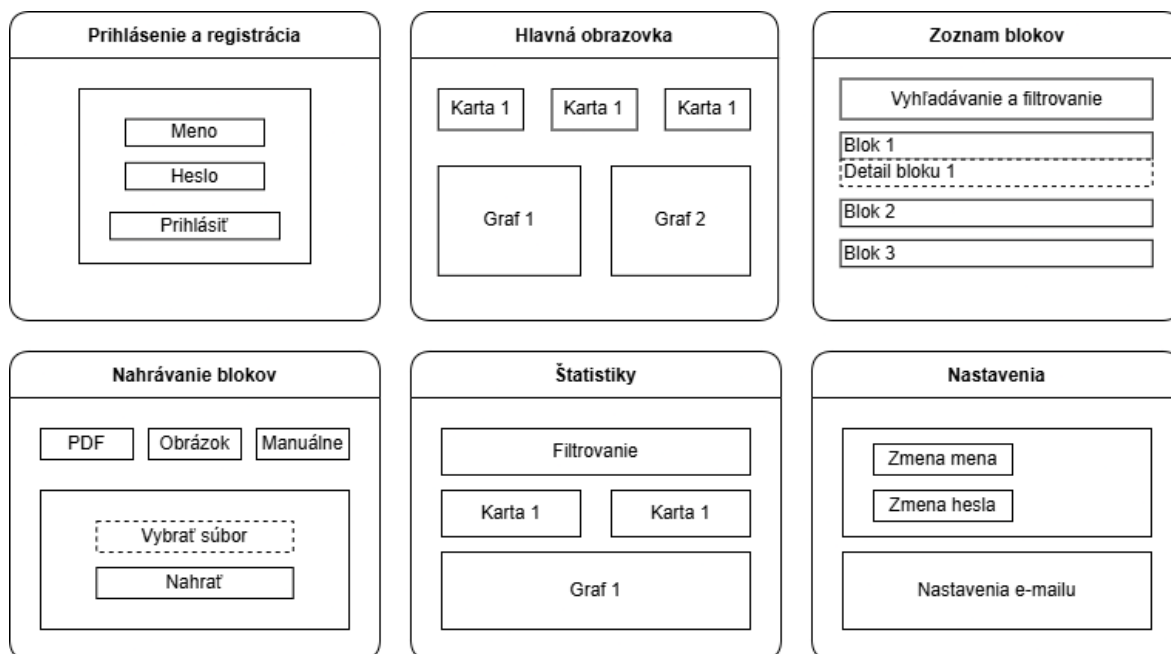
## 4.5 Návrh používateľského rozhrania

Používateľské rozhranie je implementované pomocou technológií HTML, CSS, JavaScript a knižnice Bootstrap, ktorá zabezpečuje prehľadné a responzívne zobrazenie na rôznych rozlíšeníach obrazovky.

Aplikácia je rozdelená do niekoľkých hlavných obrazoviek, ktoré pokrývajú jednotlivé funkcionality systému. Vstupným bodom je obrazovka prihlásenia a registrácie, ktorá umožňuje používateľovi prístup do systému. Po úspešnom prihlásení má používateľ k dispozícii navigačné menu, prostredníctvom ktorého prístupuje k ďalším častiam aplikácie:

- **Hlavná obrazovka** – poskytuje prehľad o výdavkoch v aktuálnom mesiaci, vrátane posledných nákupov a grafického znázornenia výdavkov.
- **Zoznam blokov** – zobrazuje všetky uložené pokladničné bloky s možnosťou filtrovania, vyhľadávania a detailného zobrazenia jednotlivých blokov.
- **Nahrávanie blokov** – umožňuje pridávanie pokladničných blokov viacerými spôsobmi, ako je nahrať PDF súboru, obrázka alebo manuálne zadanie údajov.
- **Štatistiky** – umožňuje analyzovať výdavky podľa zvoleného časového obdobia prostredníctvom grafov mesačných výdavkov, výdavkov podľa kategórií a obchodov.
- **Nastavenia** – umožňuje správu používateľského účtu, konfiguráciu e-mailového importu a nastavenie správy aplikácie.

Zjednodušený návrh jednotlivých obrazoviek je znázornený na obrázku 4.



Obr. 4: Návrh používateľského rozhrania

## 4.6 Návrh spracovania blokov

Spracovanie pokladničných blokov je navrhnuté ako hybridný proces, ktorý kombinuje offline spracovanie pomocou parserov s online spracovaním využívajúcim umelú inteligencia. Offline parsovanie nevyžaduje internetové pripojenie a zabezpečuje rýchle a deterministické spracovanie blokov podporovaných formátov a obchodných refaz-cov. V prípade zlyhania offline spracovania alebo pri spracovaní vstupov, ktoré nie je možné spracovať týmto spôsobom (napr. obrázky blokov), je využité online spracovanie prostredníctvom umelej inteligencie ako záložné riešenie.

Aplikácia podporuje štyri spôsoby importu pokladničných blokov. Najjednoduchším spôsobom je manuálny vstup, pri ktorom používateľ zadá údaje priamo do formulára. Aplikácia následne zvaliduje vyplnené údaje a uloží ich do databázy. Ďalšími spôsobmi sú nahrať PDF súboru, nahrať fotografiu pokladničného bloku a automatizovaný import z e-mailovej schránky, ktorým sa detailnejšie venujú nasledujúce podkapitoly.

Bez ohľadu na spôsob vstupu je cieľom spracovania extrahovať relevantné údaje o pokladničnom bloku a uložiť ich do databázy. Následne sú tieto údaje sprístupnené používateľovi spolu s informáciou o úspešnosti spracovania.

### 4.6.1 Spracovanie PDF bloku

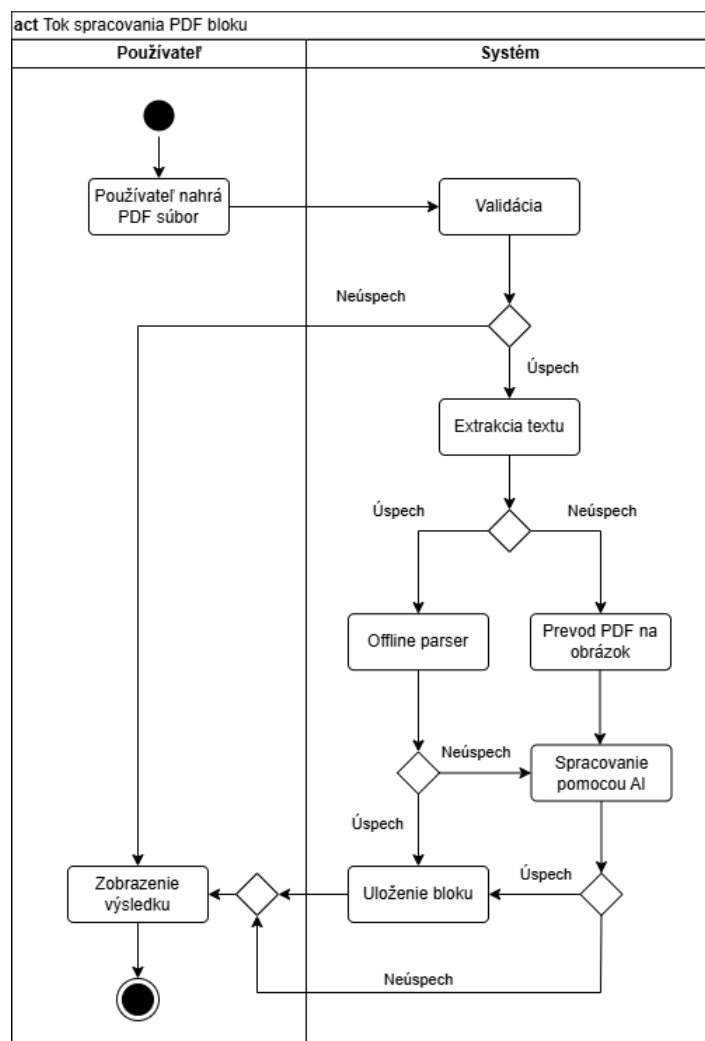
Pri nahratí PDF bloku aplikácia najprv overí formát nahratého súboru a následne sa pokúsi extrahovať jeho textový obsah. Ďalší tok spracovania závisí od výsledku tejto extrakcie.

V prípade úspešnej extrakcie textu je obsah odovzdaný offline parseru, ktorý sa pokúsi identifikovať relevantné údaje bloku, ako sú názov obchodu, dátum a čas nákupu, jednotlivé položky a ich ceny. V prípade úspešného parsovania sú tieto údaje uložené do databázy a podľa nastavení aplikácie je nahratý súbor archivovaný do priečinka súborového systému príslušného používateľa.

Offline parser môže zlyhať napríklad v prípade nepodporovanej štruktúry bloku alebo nejednoznačného formátovania položiek. V takom prípade je extrahovaný text odoslaný externému serveru, ktorý ho sprostredkuje službám umelej inteligencie. V prípade úspešného spracovania je výsledok vrátený vo forme štruktúrovanej odpovede a získané údaje sú uložené do databázy. Ak ani spracovanie pomocou umelej inteligencie nie je úspešné (napríklad pri výpadku externej služby alebo nečitateľnom dokumente), aplikácia informuje používateľa o neúspechu spracovania a blok nie je uložený do databázy.

Ak už úvodná extrakcia textu z PDF zlyhá (napríklad pri PDF dokumente obsahujúcom naskenovaný obraz namiesto textu), súbor je prevedený na obrázok a ďalej spracovaný spôsobom opísaným v nasledujúcej podkapitole.

Tok spracovania PDF bloku je znázornený na obrázku 5.



Obr. 5: Diagram aktivít – tok spracovania PDF bloku

#### 4.6.2 Spracovanie obrázku bloku

Na rozdiel od PDF blokov nie je možné z obrázku priamo extrahovať text bez dodatočného spracovania. Hoci je možné využiť technológiu OCR, jej obmedzenia boli popísané v kapitole 2 a výstup nemusí mať konzistentnú štruktúru, čo by mohlo zapríčiniť zlyhanie offline parsera. Z tohto dôvodu je obrázok odoslaný priamo externému serveru, ktorý sprostredkuje jeho spracovanie službám umelej inteligencie. Získané údaje sú uložené do databázy a podľa nastavení aplikácie je súbor archivovaný do priečinka súborového systému príslušného používateľa.

#### 4.6.3 Import z e-mailovej schránky

Automatizovaný import blokov z e-mailovej schránky umožňuje spracovanie pokladničných blokov zasielaných obchodnými reťazcami priamo na e-mailovú adresu používateľa. Aplikácia sa pripojí na e-mailovú schránku používateľa prostredníctvom protokolu

IMAP s využitím aplikačného hesla, ktoré je určené pre prístup aplikácií tretích strán. Po pripojení prehľadá správy podľa filtrov definovaných používateľom v nastaveniach aplikácie. Z identifikovaných správ sú stiahnuté PDF prílohy, ktoré sú následne spracované rovnakým spôsobom ako pri nahratí PDF súboru. Po dokončení importu je používateľovi zobrazený súhrnný prehľad výsledkov spracovaných blokov.

## 4.7 Kategorizácia položiek

Kategorizácia položiek slúži na priradenie jednotlivých položiek pokladničného bloku do kategórií, ktoré umožňujú prehľadnejšiu analýzu výdavkov.

Aplikácia najprv overí, či sa v databáze nenachádza položka s identickým názvom, ktorej už bola v minulosti priradená kategória. V takom prípade je kategória automaticky prevzatá, čím sa urýchľuje proces kategorizácie a zabezpečuje konzistentnosť opakujúcich sa položiek.

V opačnom prípade závisí ďalšie spracovanie od režimu. Pri online spracovaní prostredníctvom umelej inteligencie je položka automaticky zaradená do príslušnej kategórie na základe svojho názvu. Pri offline spracovaní je položke priradená predvolená kategória „Nezaradené“, čo zabezpečuje funkčnosť aplikácie aj bez internetového pripojenia.

Aplikácia dodatočne umožňuje používateľovi zobrazíť počet nezaradených položiek a spustiť hromadnú kategorizáciu prostredníctvom služieb umelej inteligencie.

## 4.8 Návrh externého servera

Externý server pre spracovanie AI požiadaviek je implementovaný ako samostatná komponenta pomocou frameworku FastAPI, ktorý umožňuje jednoduché vytváranie REST API rozhraní. Toto oddelenie zaisťuje, že prístupový kľúč k službám umelej inteligencie nie je súčasťou distribuovanej klientskej aplikácie.

Hlavná aplikácia odošle požiadavku na príslušný endpoint externého servera, ktorý pripraví vstupné dáta a sprostredkuje ich spracovanie modelom umelej inteligencie. Výsledok je vrátený vo forme štruktúrovanej JSON odpovede, s ktorou sa ďalej jednoducho pracuje. Server je sprístupnený prostredníctvom nástroja ngrok, ktorý umožňuje jeho použitie bez nutnosti nasadenia na verejný server. Poskytuje tri endpointy, ktorých účel a HTTP metódy sú zhrnuté v tabuľke 3.

Tabuľka 3: Endpointy externého servera

| Endpoint             | Metóda | Popis                       |
|----------------------|--------|-----------------------------|
| /parse-receipt-text  | POST   | Spracovanie textu PDF bloku |
| /parse-receipt-image | POST   | Spracovanie obrázku bloku   |
| /categorize-items    | POST   | Kategorizácia položiek      |

## 5 Implementácia aplikácie

Aplikácia pozostáva z dvoch samostatných komponentov: hlavná lokálna webová aplikácia a externý server sprostredkujúci komunikáciu so službami umelej inteligencie. Každý komponent je implementovaný ako samostatný projekt s vlastnými závislosťami a konfiguráciou, ktorých implementácia je podrobnejšie opísaná v nasledujúcich častiach kapitoly.

### 5.1 Štruktúra projektu

Hlavná aplikácia je organizovaná do modulovej štruktúry, kde sú jednotlivé súčasti rozdelené podľa ich funkčného účelu. Externá serverová komponenta tvorí samostatný projekt s jednoduchšou štruktúrou, keďže jej úlohou je výhradne sprostredkovanie komunikácie so službami umelej inteligencie. Nižšie je uvedená štruktúra oboch komponentov v jej vývojovej podobe, na ktorú v ďalšej časti nadväzuje opis distribučnej verzie.

```
Reccio/
├── app.py ..... Definícia URL adries (routes) a hlavná logika
├── config.py ..... Konfigurácia aplikácie
├── database.py ..... Inicializácia databázy
├── helper.py ..... Pomocné funkcie
├── parser.py ..... Offline parser
├── requirements.txt ..... Závislosti projektu
├── .env ..... Konfigurácia relácií a šifrovacieho kľúča
├── services/
│   ├── ai_service.py ..... Komunikácia s externým serverom
│   └── email_service.py ..... Import z e-mailu
├── static/
│   ├── css/ ..... Štýly jednotlivých stránok
│   ├── js/ ..... JavaScript súbory
│   └── fonts/ ..... Vlastné fonty
├── templates/ ..... HTML šablóny stránok
└── AI_BACKEND/
    ├── main.py ..... FastAPI endpointy
    ├── start_ai_server.bat ..... Skript na spustenie servera
    └── .env ..... API kľúč a názov modelu pre AI služby
```

Centrálnym súborom hlavnej aplikácie je `app.py`, ktorý obsahuje definície URL adries (routes) a príslušnú logiku ich spracovania pomocou frameworku Flask. Konfiguračné nastavenia aplikácie sú centralizované v súbore `config.py`, ktorý načítava hodnoty potrebné pre chod systému, ako sú cesty k súborom, databáze. Tento súbor zároveň pristupuje k citlivým údajom uloženým v súbore `.env`, konkrétne k tajnému kľúču pre správu používateľských relácií a šifrovaciemu kľúču pre uchovávanie aplikačného hesla k

e-mailovej schránke. Serverová komponenta obsahuje súbor `main.py` s implementáciou FastAPI endpointov a skript `start_ai_server.bat` na jej jednoduché spustenie.

Distribučná verzia, ktorá sa dostáva ku koncovému používateľovi, sa od vývojovej líši v zabalení klientskej aplikácie do spustiteľného súboru. Pre tento účel je využitý nástroj PyInstaller, ktorý umožňuje zabalenie aplikácie vrátane jej závislostí do jediného spustiteľného súboru bez potreby inštalácie Python prostredia na cieľovom zariadení.

Externá serverová komponenta nie je balená nástrojom PyInstaller a v distribučnom balíku zostáva v rovnakej podobe ako počas vývoja. Distribučný balík tak obsahuje dva paralelné priečinky umiestnené vedľa seba: **Reccio** so zabalenou klientskou aplikáciou a nezmenený priečinok **AI\_BACKEND** so serverovou komponentou.

```
Reccio/
├─ app.exe ..... Spustiteľný súbor aplikácie
├─ app_settings.json ..... Konfigurácia URL externého servera
└─ .env ..... Konfigurácia relácií a šifrovacieho kľúča
```

V produkčnom nasadení by externý server bežal na samostatnej serverovej infraštruktúre a klientska aplikácia by sa naň pripájala cez verejnú URL adresu. Vzhľadom na akademický charakter tejto práce a potrebu umožniť hodnotiteľom otestovať všetky funkcie aplikácie vrátane spracovania pomocou umelej inteligencie, je súčasťou distribučného balíka aj externý server. Hodnotiteľ tak môže spustiť server lokálne pomocou skriptu `start_ai_server.bat` a klientska aplikácia sa naň pripojí prostredníctvom adresy nastavenej v súbore `app_settings.json`.

Súbor `app_settings.json` obsahuje URL adresu externého servera, ktorú je možné v prípade potreby jednoducho upraviť bez zásahu do samotnej aplikácie.

Súbor `.env` v priečinku **AI\_BACKEND** obsahuje prístupový kľúč k službe umelej inteligencie a názov modelu použitého pre spracovanie požiadaviek. Tento súbor je súčasťou serverovej komponenty a klientska aplikácia k nemu nemá priamy prístup.

## 5.2 Použité knižnice

Pri implementácii boli využité knižnice zamerané na webovú vrstvu, spracovanie pokladničných blokov, komunikáciu s externými službami, bezpečné ukladanie citlivých údajov a tvorbu používateľského rozhrania. Prehľad najdôležitejších knižníc je rozdelený podľa častí systému, v ktorých sú využívané.

V hlavnej aplikácii sú použité najmä knižnice zabezpečujúce fungovanie webovej aplikácie, prácu s databázou, spracovanie PDF súborov a obrázkov a komunikáciu s externým serverom, ako je uvedené v tabuľke 4.

Tabuľka 4: Knižnice hlavnej aplikácie

| Knižnica       | Účel                                                           |
|----------------|----------------------------------------------------------------|
| Flask          | Implementácia webovej aplikácie a spracovanie HTTP požiadaviek |
| sqlite3        | Práca s lokálnou SQLite databázou                              |
| Werkzeug       | Hashovanie a overovanie hesiel                                 |
| cryptography   | Šifrovanie aplikačného hesla k e-mailu                         |
| pdfplumber     | Extrakcia textu z PDF blokov                                   |
| pypdfium2      | Prevod PDF súboru na obrázok                                   |
| Pillow         | Spracovanie obrázkov                                           |
| requests       | Odosielanie HTTP požiadaviek na externý server                 |
| python-dotenv  | Načítanie konfiguračného súboru .env                           |
| imaplib, email | Pripojenie k e-mailovej schránke a spracovanie správ           |
| base64         | Kódovanie obrázkov pri odosielaní na externý server            |

Externá serverová komponenta využíva knižnice potrebné na vytvorenie REST API rozhrania, validáciu vstupných dát a komunikáciu so službou umelej inteligencie. Ich prehľad je uvedený v tabuľke 5.

Tabuľka 5: Knižnice externého servera

| Knižnica      | Účel                                       |
|---------------|--------------------------------------------|
| FastAPI       | Framework na tvorbu REST API               |
| uvicorn       | Server na spustenie FastAPI aplikácie      |
| openai        | Komunikácia so službou umelej inteligencie |
| pydantic      | Validácia vstupných dát                    |
| python-dotenv | Načítanie konfiguračného súboru .env       |

Používateľské rozhranie využíva nasledujúce knižnice načítavané prostredníctvom CDN (Content Delivery Network), teda priamo z internetu bez potreby ich lokálnej inštalácie. Prehľad je uvedený v tabuľke 6.

Tabuľka 6: Frontend knižnice

| Knižnica        | Účel                                  |
|-----------------|---------------------------------------|
| Bootstrap       | Responzívny dizajn a UI komponenty    |
| Bootstrap Icons | Sada ikon pre používateľské rozhranie |
| SweetAlert2     | Interaktívne modálne okná             |
| Chart.js        | Vizualizácia výdavkov vo forme grafov |

### 5.3 Autentifikácia a správa používateľov

Pri registrácii sú vstupné údaje validované – overuje sa jedinečnosť používateľského mena vzhľadom na databázu, minimálna dĺžka mena a hesla a zhoda zadaných hesiel. Heslo je pred uložením do databázy zahashované pomocou funkcie `generate_password_hash` z knižnice Werkzeug:

```
password_hashed = generate_password_hash(password)
cur.execute("""
    INSERT INTO users(username, password_hashed)
    VALUES (?,?)
    """, (username, password_hashed))
con.commit()
```

Pri prihlásení je zadané heslo overené proti uloženému hashu pomocou funkcie `check_password_hash`, ktorá vstupné heslo zahashuje rovnakým spôsobom ako pri registrácii a porovná výsledok s uloženým hashom:

```
if check_password_hash(password_hashed, password):
    return True, user_id
```

Po úspešnom overení sú do relácie uložené identifikačné údaje používateľa, ako napríklad jeho ID a meno. Ak používateľ zvolí možnosť „zostať prihlásený“, relácia sa nastaví ako trvalá:

```
if remember_me:
    session.permanent = True
session['user_id'] = user_id
session['username'] = username
```

Každá chránená URL adresa (route) overuje prítomnosť identifikátora používateľa v relácii. V prípade neprihláseného používateľa dôjde k presmerovaniu na prihlasovaciu stránku:

```
if 'user_id' not in session:
    return redirect(url_for('login'))
```

Pri odhlásení sú všetky údaje relácie vymazané volaním `session.clear()` a používateľ je presmerovaný na prihlasovaciu stránku:

```
def logout():
```

```
session.clear()
return redirect(url_for('login'))
```

## 5.4 Databázová vrstva

Databáza je inicializovaná pri každom spustení aplikácie volaním funkcie `init_database()`, ktorá vytvorí tabuľky, ak ešte neexistujú.

Vzhľadom na to, že SQLite má podporu cudzích kľúčov štandardne vypnutú, je pri každom pripojení k databáze aktivovaná pomocou PRAGMA príkazu:

```
con.execute("PRAGMA foreign_keys = ON")
```

Databáza pozostáva zo štyroch tabuliek – `users`, `receipts`, `categories` a `items`. Tabuľky sú previazané cudzími kľúčmi, pričom pri vymazaní bloku sú automaticky vymazané aj všetky jeho položky vďaka kaskádovému mazaniu `ON DELETE CASCADE`. Implementácia tabuľky `items` to ilustruje nasledovne:

```
CREATE TABLE IF NOT EXISTS items(
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    item_name     TEXT NOT NULL,
    amount       REAL NOT NULL,
    price        REAL NOT NULL,
    category_id  INTEGER,
    receipt_id   INTEGER NOT NULL,
    FOREIGN KEY (receipt_id) REFERENCES receipts(id) ON DELETE CASCADE,
    FOREIGN KEY (category_id) REFERENCES categories(id)
)
```

Detekcia duplicitných blokov je zabezpečená obmedzením `UNIQUE` nad kombináciou stĺpcov `user_id`, `shop_name`, `date`, `time` a `price`. Tento spôsob je vhodnejší ako kontrola podľa názvu nahratého súboru, keďže názov súboru môže byť používateľom kedykoľvek zmenený.

V súlade s bezpečnostnými odporúčaniami uvedenými v kapitole 2.5.5 sú všetky databázové dotazy implementované ako parametrizované, čím sa eliminuje riziko SQL injection útoku, napríklad pri ukladaní údajov o položke:

```
cur.execute("""
    INSERT INTO items(item_name, amount, price,
                      category_id, receipt_id)
    VALUES (?, ?, ?, ?, ?)
    """, (item["item_name"], item["amount"],
        item["price"], category_id, receipt_id))
```

Databázový súbor je uložený v systémovom priečinku používateľa, namiesto umiestnenia vedľa spustiteľného súboru aplikácie. Týmto spôsobom sa predchádza jeho prepísaniu napríklad pri aktualizácii aplikácie. Konkrétne umiestnenie sa volí podľa operačného systému, v súlade s konvenciami pre ukladanie aplikačných dát:

```

ak operačný systém je Windows:
    cesta = LOCALAPPDATA/Reccio
inak:
    cesta = ~/.reccio
ak cesta neexistuje:
    vytvor cestu
vráť cestu

```

V prostredí Windows je to priečinok %LOCALAPPDATA%\Reccio (typicky C:\Users\<meno>\AppData\Local\Reccio) a v prostredí Linux skrytý priečinok ~/.reccio v domovskom adresári používateľa.

## 5.5 Offline spracovanie PDF

Offline spracovanie PDF blokov pozostáva z dvoch fáz – extrakcie textu a jeho parsovania. Extrakcia textu prebieha pomocou knižnice `pdfplumber`, ktorá prechádza všetky strany dokumentu a spája ich textový obsah:

```

def extract_from_pdf(path):
    text = ""
    with pdfplumber.open(path) as pdf:
        for page in pdf.pages:
            page_text = page.extract_text()
            if page_text:
                text += page_text
    return text

```

Offline spracovanie predstavuje primárny spôsob spracovania blokov, pričom nevyžaduje pripojenie na internet a zabezpečuje rýchlu odozvu aplikácie.

Z extrahovaného textu sú pomocou regulárnych výrazov identifikované hlavičkové údaje bloku – názov obchodu, dátum, čas a celková suma nákupu. Každý údaj má vlastnú funkciu – dátum je hľadaný vo formáte `DD.MM.YYYY`, čas vo formáte `HH:MM:SS` a celková suma za kľúčovým slovom ako `CELKOM` alebo `Suma`. V prípadoch, kde sa daný vzor môže v texte vyskytnúť viackrát, je možné jednoducho vytvoriť špecializovanú funkciu pre konkrétny obchodný reťazec, ktorá presnejšie definuje umiestnenie hľadaného údaju v texte.

Parsovanie položiek je navrhnuté modulárne – každý podporovaný obchodný reťazec má vlastnú funkciu parsera, ktorá je vybraná dynamicky podľa identifikovaného názvu obchodu:

```

parsers = {
    "Dr.Max" : parse_items_drmax,
    "TERNO"  : parse_items_ternio,
    "DM"     : parse_items_dm,
    "KAUFLAND": parse_items_kaufland,
    "COOP"   : parse_items_coop
}
parser = parsers.get(shopName)

```

Napriek rozdielnemu formátu blokov jednotlivých obchodných reťazcov fungujú všetky parsery na rovnakom princípe:

Vstup: PDF súbor

1. Extrahuj text zo všetkých strán PDF
2. Identifikuj hlavičkové údaje (názov obchodu, dátum, čas, suma)
3. Podľa názvu obchodu zvol príslušný parser položiek
4. V parseri:
  - a. Nájdi začiatok sekcie položiek podľa vzoru
  - b. Nájdi koniec sekcie položiek podľa vzoru
  - c. Ak začiatok alebo koniec nenájdený -> vráť prázdny zoznam
  - d. Pre každý riadok medzi začiatkom a koncom:
    - > ak obsahuje vzor položky: ulož názov, množstvo a cenu do zoznamu
    - > ak obsahuje zľavu alebo bonus: preskoč
5. Vráť zoznam položiek

Po úspešnom parsovaní sú údaje uložené do databázy. Následne je na základe nastavení používateľa rozhodnuté o archivácii pôvodného súboru, ktorej mechanizmus je podrobnejšie opísaný v kapitole 5.9.

Vďaka modulárnej architektúre je možné pridať podporu nového obchodného reťazca doplnením novej funkcie parsera a jej registráciou do slovníka. V prípade zmeny formátu bloku stačí vykonať minimálne úpravy regulárnych výrazov alebo doplniť jednoduchú funkciu bez zásahu do ostatných častí aplikácie.

V prípade, že extrakcia textu zlyhá alebo offline parser neidentifikuje podporovaný formát bloku, spracovanie pokračuje prostredníctvom umelej inteligencie, čomu sa venuje nasledujúca časť.

## 5.6 Spracovanie pomocou umelej inteligencie

Spracovanie pomocou umelej inteligencie nastáva v troch prípadoch – ak offline parser neidentifikuje podporovaný formát bloku, ak sa nepodarí extrahovať text z PDF súboru, alebo ak používateľ nahrať fotografiu pokladničného bloku.

V prípade úspešnej extrakcie textu z PDF, pri ktorej však offline parser zlyhal, je text odoslaný na endpoint `/parse-receipt-text`. Ak extrakcia textu zlyhá, súbor je prevedený na obrázok a odoslaný na endpoint `/parse-receipt-image`. Prevod na obrázok je nevyhnutným krokom daným povahou multimodálnych jazykových modelov, ktoré prijímajú obrazový vstup a nie binárny formát PDF. Rovnaký endpoint je využitý aj pri priamom nahrať fotografie bloku používateľom. Dočasný obrázok vytvorený prevodom z PDF je po odoslaní zo súborového systému odstránený.

Dôležitým rozdielom oproti offline spracovaniu je spôsob kategorizácie položiek. Pri spracovaní pomocou umelej inteligencie je kategória vrátená priamo ku každej položke v odpovedi servera. Pri offline spracovaní sa aplikácia pokúsi nájsť zhodu s rovnako

pomenovanou položkou v existujúcich záznamoch používateľa. Ak zhoda neexistuje, položka zostane nezaraďená.

Okrem spracovania blokov zabezpečuje aplikácia aj hromadnú kategorizáciu nezaraďených položiek. Keďže pri väčšom počte položiek by jedno volanie obsahovalo neúmerne veľa dát, zoznam je pred odoslaním rozdelený do menších skupín, ktoré sú spracované samostatnými volaniami.

## 5.7 Externý server

Externý server je implementovaný ako samostatná komponenta pomocou frameworku FastAPI a slúži ako prostredník medzi hlavnou aplikáciou a službami umelej inteligencie. Toto oddelenie zabezpečuje, že prístupový kľúč k OpenAI API nie je súčasťou distribuovanej klientskej aplikácie.

Každý z troch endpointov najprv overí prítomnosť konfiguračných premenných – prístupového kľúča a názvu modelu, ktoré sú načítané zo súboru `.env`. Endpointy `/parse-receipt-text` a `/parse-receipt-image` prijímajú text, resp. obrázok bloku a pomocou vopred definovaného promptu navedú model na vrátenie štruktúrovaného JSON výstupu obsahujúceho názov obchodu, dátum, čas, celkovú sumu a zoznam položiek vrátane ich kategórií. Endpoint `/categorize-items` prijíma zoznam položiek a pre každú z nich určí kategóriu z pevne definovanej množiny: *Potraviny*, *Drogéria*, *Lieky*, *Elektronika*, *Oblečenie*, *Ostatné*.

Server je sprístupnený prostredníctvom nástroja ngrok, ktorý generuje dynamickú verejnú URL adresu pri každom spustení. Keďže sa táto adresa mení, je konfigurovateľná v súbore `app_settings.json` klientskej aplikácie bez potreby zásahu do jej zdrojového kódu.

## 5.8 Import z e-mailu

Automatizovaný import blokov z e-mailovej schránky je implementovaný pomocou protokolu IMAP so zabezpečeným SSL pripojením na server `imap.gmail.com`. Prihlásenie prebieha pomocou aplikačného hesla, ktoré je uložené v databáze v šifrovanej forme.

Proces importu prebieha podľa nasledujúceho algoritmu:

1. Pripoj sa na Gmail server cez IMAP SSL
2. Načítaj posledných N správ podľa limitu používateľa
3. Pre každú správu:
  - a. Načítaj odosielateľa a predmet správy
  - b. Ak odosielateľ ani predmet neobsahuje žiadny z filtrov používateľa -> preskoc správu
  - c. Pre každú prílohu správy:
    - > ak nie je vo formáte PDF -> preskoc
    - > ak príloha presahuje maximálnu veľkosť -> preskoc

- > ulož prílohu ako dočasný súbor
  - > spracuj PDF rovnakým spôsobom ako pri nahratí  
PDF súboru používateľom (kapitola 4.6.1)
  - > zaznamenaj výsledok: importovaná / duplicitná / chyba
4. Odpoj sa od servera
  5. Vráť súhrnný prehľad výsledkov

Filtre definované používateľom v nastaveniach aplikácie predstavujú textové reťazce porovnávané s adresou odosielateľa a predmetom každej správy. Zhoda nastáva v prípade, že niektorý z filtrov je obsiahnutý v adrese odosielateľa alebo predmete správy. Takýmto filtrom môže byť napríklad e-mailová adresa obchodného reťazca alebo kľúčové slovo v predmete správy.

Každá spracovaná príloha je označená jedným z troch výsledkov. Výsledok *importovaná* znamená, že blok bol úspešne spracovaný a uložený do databázy, pričom podľa nastavenia používateľa je súbor archivovaný aj do lokálneho priečinka na disku. Výsledok *duplicitná* znamená, že aplikácia na základe detekcie duplícít zistila, že daný blok sa už v databáze nachádza a neuložila ho znova. Výsledok *neúspešne spracovaná* nastáva v prípade, že sa nepodarilo extrahovať údaje z prílohy ani offline parserom, ani spracovaním pomocou umelej inteligencie. Po dokončení importu je používateľovi zobrazený súhrnný prehľad výsledkov všetkých spracovaných príloh.

## 5.9 Archivácia súborov

Archivácia súborov je voliteľná funkcia konfigurovateľná v nastaveniach používateľa, ktorá sa uplatňuje rovnako pri všetkých spôsoboch importu – nahratí PDF súboru, nahratí fotografie aj automatizovanom importe z e-mailovej schránky.

Ak je archivácia povolená, súbor je premenovaný podľa identifikátora bloku v databáze, čím sa zaručuje jedinečnosť názvu súboru, a uložený do priečinka `uploads/user{id}` príslušného používateľa na lokálnom disku. Cesta k súboru je zároveň uložená do databázy, čo umožňuje jeho následné zobrazenie priamo z rozhrania aplikácie. V opačnom prípade je dočasný súbor po spracovaní odstránený, čím sa predchádza zbytočnému zaberaniu úložného priestoru:

```
Ak je ukladanie súborov zapnuté:
    Premenuj súbor podľa identifikátora bloku
    Ulož cestu k súboru do databázy
Inak:
    Vymaž dočasný súbor
```

## 5.10 Štatistiky a grafy

Štatistiky sú dostupné na dvoch miestach v aplikácii. Hlavná obrazovka zobrazuje prehľad aktuálneho mesiaca – celkovú sumu výdavkov, počet nákupov, priemernú sumu

nákupu a najnavštevovanejší obchod. Doplnené sú o graf denných výdavkov aktuálneho mesiaca, koláčový graf rozloženia výdavkov podľa kategórií a zoznamy posledných piatich a najdrahších piatich nákupov.

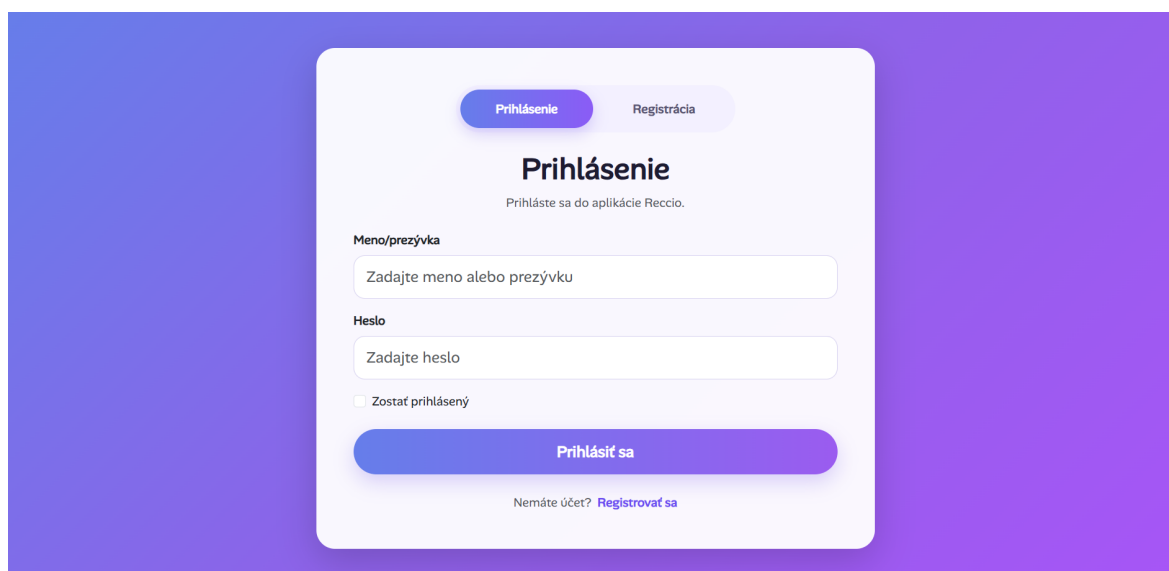
Stránka štatistík umožňuje filtrovanie podľa ľubovoľného časového obdobia a zobrazuje celkovú sumu výdavkov, počet blokov, najčastejšie kupovanú kategóriu a najčastejšie kupovanú položku za zvolené obdobie. Doplnené sú o graf mesačných výdavkov, graf výdavkov podľa kategórií a graf výdavkov podľa obchodov, pričom pri grafe obchodov je zobrazených najviac šesť obchodov s najvyššími výdavkami. Všetky grafy sú vykreslené pomocou knižnice Chart.js.

Štatistické údaje sú vypočítavané priamo SQL dotazmi nad databázou a odovzdané do HTML šablón, kde sú následne vykreslené.

## 5.11 Používateľské rozhranie

Používateľské rozhranie vychádza z návrhu opísaného v kapitole 4.5. Responzívne zobrazenie je zabezpečené knižnicou Bootstrap, interaktívne dialógové okná knižnicou SweetAlert2 a grafy knižnicou Chart.js. Aplikácia využíva vlastné fonty AlanSans a Boldonse.

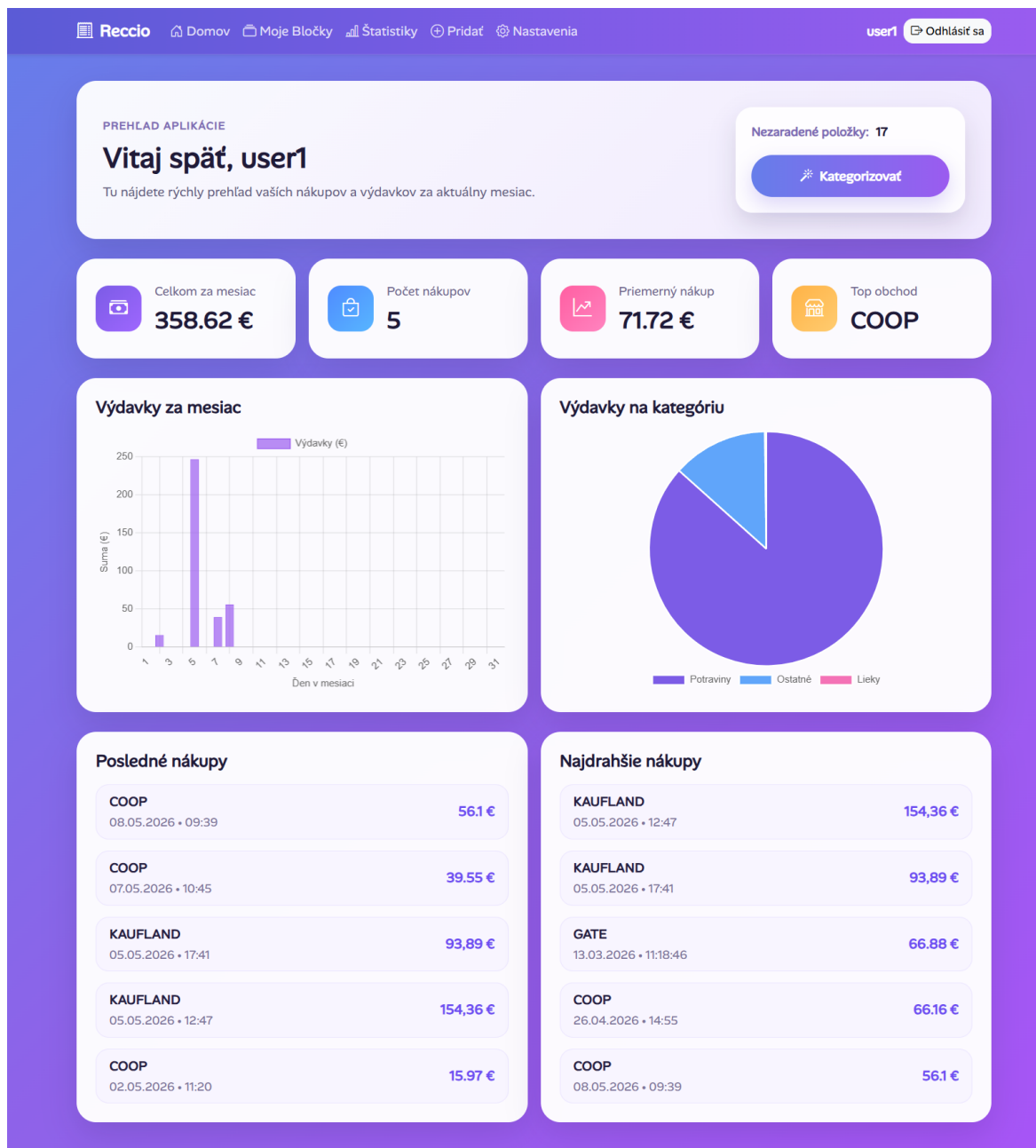
Vstupným bodom aplikácie je obrazovka prihlásenia a registrácie, znázornená na obrázku 6. Prepínanie medzi prihlásením a registráciou je realizované prepínačom v hornej časti obrazovky. Obrazovka obsahuje aj možnosť zostať prihlásený, ktorá nastaví reláciu ako trvalú.



Obr. 6: Obrazovka prihlásenia a registrácie

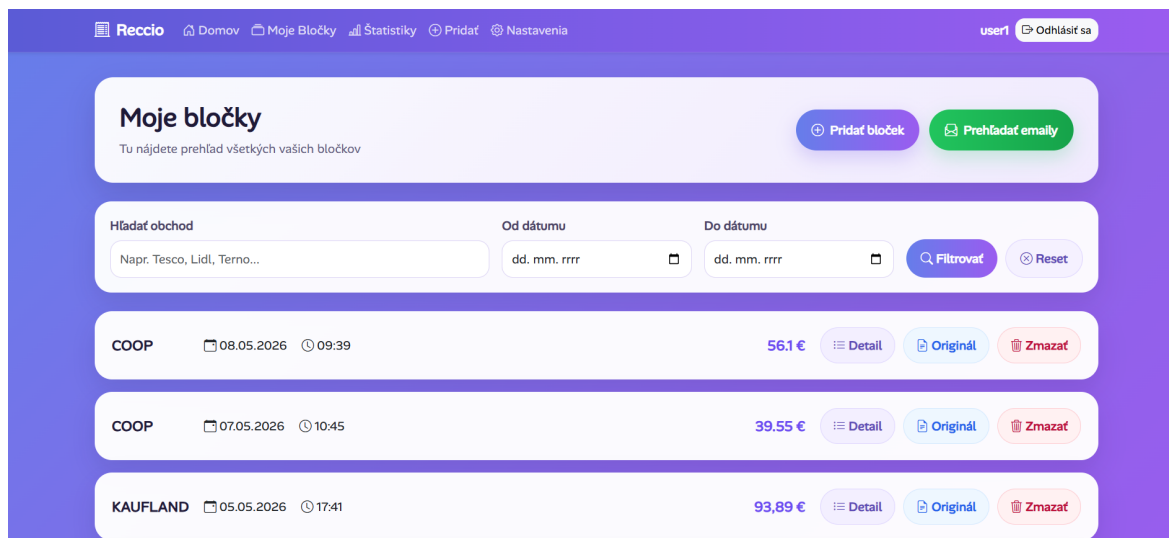
Po prihlásení je používateľovi dostupné navigačné menu, ktoré je spoločnou súčasťou všetkých obrazoviek aplikácie. Obsahuje odkazy na hlavnú obrazovku, zoznam blokov, štatistiky, pridanie bloku a nastavenia. V pravej časti menu je zobrazené meno prihláseného používateľa a tlačidlo na odhlásenie.

Hlavná obrazovka, znázornená na obrázku 7, poskytuje používateľovi prehľad aktuálneho mesiaca. Horná časť zobrazuje štyri sumarizačné kartičky – celkovú sumu výdavkov, počet nákupov, priemernú sumu nákupu a najnavštevovanejší obchod. Zároveň je tu zobrazený počet nezaradených položiek s možnosťou spustenia hromadnej kategorizácie. Spodná časť obsahuje graf denných výdavkov, koláčový graf rozloženia výdavkov podľa kategórií a zoznamy posledných piatich a najdrahších piatich nákupov.



Obr. 7: Hlavná obrazovka

Obrazovka zoznamu blokov, znázornená na obrázku 8, zobrazuje všetky uložené pokladničné bloky zoradené od najnovšieho. Pre každý blok je zobrazený názov obchodu, dátum, čas a celková suma. Obrazovka umožňuje filtrovanie blokov podľa názvu obchodu a časového obdobia. Pri každom bloku sú dostupné tri akcie – zobrazenie detailu položiek, zobrazenie originálneho súboru a vymazanie bloku. Z tejto obrazovky je taktiež možné spustiť automatizovaný import blokov z e-mailovej schránky.



Obr. 8: Obrazovka zoznamu blokov

Obrazovka štatistík umožňuje filtrovanie podľa ľubovoľného časového obdobia a zobrazuje celkovú sumu výdavkov, počet blokov, najčastejšiu kategóriu a najčastejšie kupovanú položku za zvolené obdobie, doplnené o grafy opísané v kapitole 5.10.

Obrazovka nahrávania blokov umožňuje pridávanie pokladničných blokov troma spôsobmi – nahratím súboru vo formáte PDF, nahratím fotografie bloku a manuálnym zadaním údajov. Každý spôsob je dostupný prostredníctvom samostatnej záložky. Pri manuálnom zadaní používateľ vyplní formulár s údajmi o obchode, dátume, čase, celkovej sume a jednotlivých položkách.

Obrazovka nastavení umožňuje správu používateľského účtu – zmenu používateľského mena a hesla. Samostatná sekcia je venovaná konfigurácii e-mailového importu, kde používateľ zadáva Gmail adresu, aplikačné heslo, filtre pre filtrovanie správ, limit počtu prehľadávaných správ a voliteľnú archiváciu súborov na disk.

## 6 Testovanie a vyhodnotenie

6.1 Ciele a prostredie testovania

6.2 Testovanie funkcií aplikácie

6.3 Testovanie na reálnych bločkoch

6.4 Vyhodnotenie výsledkov

6.5 Porovnanie s existujúcimi riešeniami

6.6 Obmedzenia riešenia

6.7 Možnosti rozšírenia

# Záver

*„V tejto príručke sme sa snažili poskytnúť praktické pokyny pre vypracovanie záverečných prác na Fakulte elektrotechniky a informatiky STU v Bratislave. Obsahuje dôležité informácie o formálnej stránke dokumentu, ktoré sú nevyhnutné pre zabezpečenie jasnej štruktúry a správneho formátovania práce. Naša príručka kladie dôraz na dodržiavanie základných pravidiel, ktoré pomáhajú minimalizovať chyby a zlepšujú celkovú kvalitu predkladaných dokumentov.*

*Rovnakú pozornosť venujeme aspektom, ako sú formátovanie textu, zarovnávanie, riadkovanie a použitie preddefinovaných štýlov, ktoré prispievajú k profesionálnemu vzhľadu dokumentu. Dôkladné spracovanie matematických rovníc a grafov je ďalším dôležitým prvkom, ktorý môže ovplyvniť hodnotenie práce. Práca s týmito aspektmi zaisťuje, že obsah je prehľadný a presne odráža zamýšľané myšlienky.*

*V neposlednom rade sa naša príručka zameriava na význam citovania externých zdrojov, čo je zásadné pre zachovanie integrity a etických štandardov pri písaní akademických textov. Správne citovanie nielenže posilňuje argumentáciu, ale tiež pomáha predchádzať plagiátorstvu, ktoré môže mať vážne následky na akademickú kariéru.*

*Dodržiavanie týchto pokynov umožňuje vyhnúť sa bežným chybám a zvyšuje kvalitu záverečných prác, čím sa zlepšuje celkový dojem z akademických schopností. Dobre vypracovaná práca môže viesť k pozitívnemu hodnoteniu a uznaniu zo strany hodnotiteľov, čo je cieľom každého študenta.*

*Celkovo je príručka navrhnutá ako cenný nástroj na podporu pri písaní záverečných prác. Poskytnuté informácie a odporúčania prispejú k dosiahnutiu akademických cieľov a pomôžu vypracovať kvalitné a odborné dokumenty, ktoré budú spĺňať štandardy fakulty.“*

*(ChatGPT 3, 2024)*

Poznámka autora: Časť záveru vygenerovala umelá inteligencia na základe analýzy obsahu verzie dokumentu pre práce písané v MS Word. Konkrétne išlo o aplikáciu ChatGPT 3 dňa 29. 9. 2024. Rukopis umelej inteligencie je z textu cítiť aj napriek tomu, že celá komunikácia vedúca k tejto forme záveru trvala približne 40 minút. Parafrázujúc ľudové príslovie konštatujeme, že umelá inteligencia je možno dobrý sluha, ale zlý pán. Používajme ju s rozumom.

Pripomíname, že takýto spôsob tvorby obsahu je v záverečnej práci, ako aj v iných akademických textoch, neprípustný. Autorské dielo nesmie obsahovať odseky vytvorené generatívnou umelou inteligenciou. Preto sme ho formátovali ako citovaný text.

# Literatúra

1. FISKALY. *Digital receipt automation and consumer centricity* [online]. 2024. [cit. 2026-04-09]. Dostupné z: <https://www.fiskaly.com/blog/digital-receipt-automation-and-consumer-centricity-2024>.
2. GREEN AMERICA. *Skip the Slip: The Big Impacts of Paper Receipts* [online]. 2022. [cit. 2026-04-09]. Dostupné z: <https://reports.greenamerica.org/skip-the-slip>. Version 4, published September 7, 2022.
3. NÁRODNÁ RADA SLOVENSKEJ REPUBLIKY. *Zákon č. 289/2008 Z. z. o používaní elektronickej registračnej pokladnice*. 2008. Dostupné tiež z: <https://www.zakonypreludi.sk/zz/2008-289>. Accessed: 2026-04-09.
4. FRANKOWSKI, R.; ZGOŁA-GRZEŠKOWIAK, A.; GRZEŠKOWIAK, T.; SÓJKA, K. The presence of bisphenol A in thermal paper in the face of changing European regulations – A comparative global research. *Environmental Pollution*. 2020, **265**, 114879. Dostupné z DOI: 10.1016/j.envpol.2020.114879.
5. PLAZONIĆ, I.; BARBARIĆ-MIKOČEVIĆ, Ž.; KRALJ, M. Stability of the cash register receipt depending on the thermal paper used. *Innovations in Printing, Publishing and Multimedia Technologies*. 2024, 78–87. Dostupné z DOI: 10.59476/ilpmt2024.78-87.
6. ASRIANI; SAID, M.; ARNOUS, A. H. Digital Receipts for a Greener Future: Web-Based Innovation in Toll Payment Systems. *Ceddi Journal of Information System and Technology (JST)*. 2025, **4**(1), 22–30. Dostupné z DOI: 10.56134/jst.v4i1.79.
7. ISO. *Document management – Portable document format – Part 2: PDF 2.0*. 2020. Tech. spr., ISO 32000-2:2020. International Organization for Standardization. Dostupné tiež z: <https://www.iso.org/standard/75839.html>.
8. PYTHON SOFTWARE FOUNDATION. *re — Regular expression operations*. 2024. Dostupné tiež z: <https://docs.python.org/3/library/re.html>. Accessed: 2026-04-18.
9. MEMON, J.; SAMI, M.; KHAN, R. A.; UDDIN, M. Handwritten Optical Character Recognition (OCR): A Comprehensive Systematic Literature Review (SLR). *IEEE Access*. 2020, **8**, 142642–142668. Dostupné z DOI: 10.1109/ACCESS.2020.3012542.
10. TAVARES, R. A. Comparison of Image Preprocessing Techniques for Vehicle License Plate Recognition Using OCR: Performance and Accuracy Evaluation. 2024. Dostupné tiež z: <https://arxiv.org/abs/2410.13622>.

11. MINAE, S. et al. Large Language Models: A Survey. 2024. Dostupné tiež z: <https://arxiv.org/abs/2402.06196>.
12. YIN, S. et al. A survey on multimodal large language models. *National Science Review*. 2024, **11**(12). Dostupné z DOI: 10.1093/nsr/nwae403.
13. VILLENA-ROMÁN, J.; COLLADA-PÉREZ, S.; LANA-SERRANO, S.; GONZÁLEZ-CRISTÓBAL, J. C. Hybrid Approach Combining Machine Learning and a Rule-Based Expert System for Text Categorization. In: *Proceedings of the 24th International FLAIRS Conference* [online]. AAAI Press, 2011, s. 323–328 [cit. 2026-04-09]. Dostupné z: <https://cdn.aaai.org/ocs/2532/2532-11166-1-PB.pdf>.
14. PYTHON SOFTWARE FOUNDATION. *What is Python? Executive Summary*. 2024. Dostupné tiež z: <https://www.python.org/doc/essays/blurb/>. Accessed: 2026-04-18.
15. PALLETS PROJECTS. *Flask Documentation*. 2024. Dostupné tiež z: <https://flask.palletsprojects.com/>. Accessed: 2026-04-18.
16. SEBASTIÁN RAMÍREZ. *FastAPI Documentation*. 2024. Dostupné tiež z: <https://fastapi.tiangolo.com/>. Accessed: 2026-04-18.
17. MOZILLA FOUNDATION. *HTML: HyperText Markup Language*. 2024. Dostupné tiež z: <https://developer.mozilla.org/en-US/docs/Web/HTML>. Accessed: 2026-04-18.
18. MOZILLA FOUNDATION. *CSS: Cascading Style Sheets*. 2024. Dostupné tiež z: <https://developer.mozilla.org/en-US/docs/Web/CSS>. Accessed: 2026-04-18.
19. MOZILLA FOUNDATION. *JavaScript*. 2024. Dostupné tiež z: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. Accessed: 2026-04-18.
20. BOOTSTRAP TEAM. *Bootstrap Documentation*. 2024. Dostupné tiež z: <https://getbootstrap.com/docs/>. Accessed: 2026-04-18.
21. SQLITE CONSORTIUM. *About SQLite*. 2024. Dostupné tiež z: <https://www.sqlite.org/about.html>. Accessed: 2026-04-18.
22. SQLITE CONSORTIUM. *Appropriate Uses For SQLite*. 2024. Dostupné tiež z: <https://www.sqlite.org/whentouse.html>. Accessed: 2026-04-18.
23. CRISPIN, M. *Internet Message Access Protocol – Version 4rev1*. 2003. Tech. spr., RFC 3501. Internet Engineering Task Force (IETF). Dostupné tiež z: <https://datatracker.ietf.org/doc/html/rfc3501>.

24. FIELDING, R. T. *Architectural Styles and the Design of Network-based Software Architectures*. 2000. Dostupné tiež z: [https://roy.gbiv.com/pubs/dissertation/fielding\\_dissertation.pdf](https://roy.gbiv.com/pubs/dissertation/fielding_dissertation.pdf). Diz. pr. University of California, Irvine. Accessed: 2026-04-18.
25. FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. *HTTP Semantics* [online]. 2022-06. [cit. 2026-04-09]. Tech. spr., RFC 9110. Internet Engineering Task Force. Dostupné z DOI: 10.17487/RFC9110.
26. NGROK INC. *What is ngrok?* 2024. Dostupné tiež z: <https://ngrok.com/docs/what-is-ngrok>. Accessed: 2026-04-18.
27. NGROK INC. *How ngrok works?* 2024. Dostupné tiež z: <https://ngrok.com/docs/how-ngrok-works>. Accessed: 2026-04-18.
28. OWASP FOUNDATION. *Password Storage Cheat Sheet*. 2024. Dostupné tiež z: [https://cheatsheetseries.owasp.org/cheatsheets/Password\\_Storage\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html). Accessed: 2026-04-18.
29. AUTH0. *Hashing Passwords: One-Way Road to Security*. 2021. Dostupné tiež z: <https://auth0.com/blog/hashing-passwords-one-way-road-to-security/>. Accessed: 2026-04-18.
30. NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. *An Introduction to Computer Security: The NIST Handbook, Chapter 19 – Cryptography*. 1995. Dostupné tiež z: <https://csrc.nist.rip/publications/nistpubs/800-12/800-12-html/chapter19.html>. Accessed: 2026-04-18.
31. OWASP FOUNDATION. *Session Management Cheat Sheet*. 2024. Dostupné tiež z: [https://cheatsheetseries.owasp.org/cheatsheets/Session\\_Management\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html). Accessed: 2026-04-18.
32. OWASP FOUNDATION. *Input Validation Cheat Sheet*. 2024. Dostupné tiež z: [https://cheatsheetseries.owasp.org/cheatsheets/Input\\_Validation\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html). Accessed: 2026-04-18.
33. OWASP FOUNDATION. *SQL Injection Prevention Cheat Sheet*. 2024. Dostupné tiež z: [https://cheatsheetseries.owasp.org/cheatsheets/SQL\\_Injection\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html). Accessed: 2026-04-18.
34. EBLOČKY. *eBločky – aplikácia na správu bločkov*. 2024. Dostupné tiež z: <https://www.eblocky.sk/>. Accessed: 2026-04-18.

35. FINANČNÁ SPRÁVA SR. *Pokladničné doklady musia obsahovať jedinečný QR kód*. 2020. Dostupné tiež z: [https://www.financnasprava.sk/sk/pre-media/novinky/archiv-noviniek/detail-novinky/\\_ekasa-blocky-over-ts/bc](https://www.financnasprava.sk/sk/pre-media/novinky/archiv-noviniek/detail-novinky/_ekasa-blocky-over-ts/bc). Accessed: 2026-04-18.
36. E-BLOK. *E-blok – aplikácia pre evidenciu nákupov*. 2024. Dostupné tiež z: <https://e-blok.sk/>. Accessed: 2026-04-18.
37. COOP JEDNOTA. *E-doklad*. 2024. Dostupné tiež z: <https://coop.sk/sk/e-doklad>. Accessed: 2026-04-18.
38. TERNNO. *Terno karta*. 2024. Dostupné tiež z: <https://terno.sk/terno-karta>. Accessed: 2026-04-18.
39. DR.MAX. *Zelená pre planetu*. 2024. Dostupné tiež z: <https://www.drmax.sk/zelená-pre-planetu>. Accessed: 2026-04-18.
40. EXPENSIFY INC. *Expensify: Spend Management Software*. 2024. Dostupné tiež z: <https://www.expensify.com/>. Accessed: 2026-04-18.
41. REACTIVE APPS CORP. *Smart Receipts: Expense Tracker*. 2024. Dostupné tiež z: <https://smartreceipts.co/>. Accessed: 2026-04-18.

## Použitie nástrojov umelej inteligencie

## Dodatok A: Algoritmus

## Dodatok B: Výpis dlhého kódu

## Dodatok C: Slovníček pojmů